

技术白皮书

workbuddy2api-plus-cf

Cloudflare 版架构 · 实现路径 · 技术流全解

文档类型：架构审计与技术交底

覆盖范围：118 文件 / 约 28,396 行源码

生成日期：2026-10-07

目 录

workbuddy2api-plus-cf 技术白皮书

第一部分：项目定性

- 1.1 一句话定义
- 1.2 基本画像
- 1.3 核心业务能力

第二部分：系统架构

- 2.1 架构总览
- 2.2 为什么必须是两个部署单元
- 2.3 为什么两件事合在同一个 Worker 里
- 2.4 请求流转路径（完整链路）

第三部分：目录与代码结构

- 3.1 完整目录树
- 3.2 分层职责

第四部分：核心实现路径

- 4.1 账号池（PoolDO）
- 4.2 错误分类（classify.ts）
- 4.3 模型编排（autoroute.ts）
- 4.4 反向代理轮转循环（proxy.ts）
- 4.5 SSE 流式处理（sse.ts）
- 4.6 子密钥体系（apikey.ts）
- 4.7 任务中心（tasks.ts，902 行）
- 4.8 上游通信（upstream.ts，1191 行）

第五部分：可观测性体系

- 5.1 分段计时（timing.ts）
- 5.2 冷启动观测（boot.ts）
- 5.3 联动读法（实战判读）
- 5.4 CORS 与 Expose-Headers

第六部分：Go → Cloudflare 迁移映射

- 6.1 状态外移总表
- 6.2 依赖替代表
- 6.3 模块映射表（部分关键项）
- 6.4 前端零改动的保障机制
- 6.5 CLI 的云原生重构

第七部分：部署路径

- 7.1 三条部署路线
- 7.2 零配置部署的核心技巧
- 7.3 部署顺序（不可颠倒）
- 7.4 D1 自动迁移（无需手工步骤）
- 7.5 部署校验三件套
- 7.6 fill-ids.mjs（资源 ID 注入）
- 7.7 两条定时驱动路径
- 7.8 定时作业时刻表
- 7.9 已知环境限制

第八部分：工程实践与踩坑档案

- 8.1 三类「本地测不出」的问题
- 8.2 具体的真机 bug 修复档案
- 8.3 工程习惯总结

第九部分：测试与质量保障

- 9.1 测试策略
- 9.2 测试命名反映的关注点
- 9.3 pool-core.test.ts 用例清单（32 例）

第十部分：设计取舍总表

- 10.1 关键参数及其「经营含义」
- 10.2 技术选型取舍
- 10.3 「宁可…也不…」清单

第十一部分：架构张力与遗留问题

- 11.1 核心张力：跨 Worker RPC 成本
- 11.2 total 与各段之和的差值
- 11.3 已识别的残余风险
- 11.4 待确认的设计决策（来自审计文档）
- 11.5 最有复用价值的五个文件
- 11.6 一句话总结

workbuddy2api-plus-cf 技术白皮书

一份完整的架构、实现与工程实践分析 分析对象: <https://github.com/Linyueying/workbuddy2api-plus-cf> 分析基
准: `main` 分支 @ `fe0867d` (2026-10-06), 共 60 次提交 文档性质: 基于真实源码的逐层剖析, 非推测

第一部分：项目定性

1.1 一句话定义

workbuddy2api-plus-cf 是一个把 Go 版多账号 OpenAI 兼容网关完整重写为 Cloudflare 云原生架构的项目。

它不是逐行翻译，而是云原生重写：

原 Go 的「进程内存状态 + 本地文件 + 常驻 goroutine 调度器」整体外移到 **Durable Objects / D1 / KV / R2**。前端（相对路径 + Bearer 鉴权）**零改动**部署。

1.2 基本画像

维度	数值
语言	TypeScript (ESM)
框架	Hono 4.6 (Pages Functions 高级模式)
运行时	Cloudflare Workers / Pages
打包	esbuild 0.24 (--bundle --format=esm --minify)
测试	Vitest 2.1 + Miniflare 3
后端源码	12,817 行 TS (src/ + engine-worker/src/)
前端	4,730 行 (vendor/frontend/ : app.js 3,098 + index.html 1,632)
测试	8,363 行 / 33 个文件 / 约 680 个用例
脚本	1,289 行 (10 个 .mjs)
提交	60 次 (dev 56 / workbuddy-agent 4)
分支	main / dev
时间跨度	2026-10-02 ~ 2026-10-06

1.3 核心业务能力

这是一个**有状态**的多账号网关，对外提供 OpenAI 兼容接口，对内管理一批上游账号：

能力域	说明
协议兼容	OpenAI Chat / OpenAI Responses / Anthropic Messages 三套入口

能力域	说明
多账号池	加权选号、四层冷却、熔断、在途限流、会话粘性
token 生命周期	自动刷新、保活、OAuth 设备流登录
流式转发	SSE 逐帧规范化透传 / 非流式聚合
错误治理	13 类错误保序分类 + 差异化惩罚策略 + 模型降级链
成本优化	前缀缓存键注入、成本分层选号、积分保底
活动任务	22 个任务码的事件链编排（签到/旅行/夜猫子/成长等）
管理面板	8 视图 + 2 页签，零框架原生 JS
观测体系	Server-Timing 十段计时 + 4 个自定义诊断头

第二部分：系统架构

2.1 架构总览

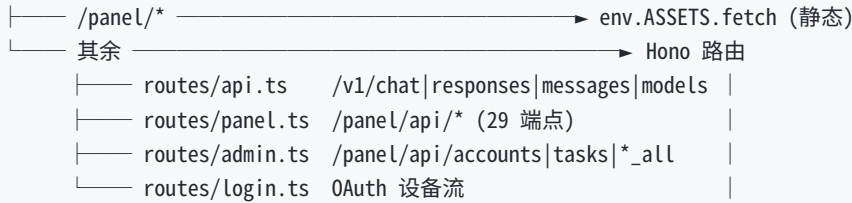
客户端

- ① OpenAI 兼容客户端 (/v1/*) ② 浏览器面板 (/panel/)
- ③ CLI 瘦客户端 (scripts/wb2api.mjs)



① workbuddy2api-pages (Cloudflare Pages)

入口 _worker.js (Hono 高级模式)



中间件链: 计时起点 → CORS → 自动迁移 → 鉴权 → 路由

服务层 services/ (30 文件): proxy|sse|upstream|classify|...

存储层 storage/ (7 文件): kv|d1|r2|migrate|reqlog|...

| 跨 Worker RPC: env.POOL.get("main").fetch()



② workbuddy2api-engine (独立 Worker)

唯一目的: 补 Pages 的两处平台空缺

PoolDO (账号池 Durable Object, 单实例)

选号 / 四层冷却 / 熔断 / 在途 / 会话粘性

状态存 DO Storage (SQLite 后端)

Cron Triggers

"0 * * * *" 整点作业 (签到/旅行/活跃/保活/夜猫子/成长)

"30 17 * * *" UTC 17:30 低峰: D1 日志 → R2 归档



资源层

KV ×2 (WB2A_CONFIG 配置 / WB2A_CACHE 模型缓存)

D1 (WB2A_DB 密钥/日志/用量/队列, 4 表 5 索引)

R2 (WB2A_LOGS 日志归档, 可选)

DO (POOL PoolDO)



上游 Copilot 服务

cn: chat=copilot.tencent.com billing=codebuddy.cn

```
| global: chat=billing=web=workbuddy.ai |
```

2.2 为什么必须是两个部署单元

这不是设计选择，而是**平台硬约束**。四条规则互相锁死：

规则	来源	后果
DO 类必须靠 <code>[[migrations]]</code> 才能注册	Workers 运行时（删掉则 DO 调用挂起）	—
Pages 的 <code>wrangler.toml</code> 不支持 <code>migrations</code>	云端构建报 <code>does not support "migrations"</code>	Pages 无法自带 DO
Pages 的 DO binding 强制要求 <code>script_name</code>	云端构建报 <code>should specify a "script_name"</code>	必须远程引用
Pages 没有 Cron Triggers	平台约束	定时作业无处安放

官方文档原文："You cannot create and deploy a Durable Object within a Pages project"

解法：`PoolDO` 放在 `engine-worker/`，Pages 侧用 `script_name = "workbuddy2api-engine"` 远程引用。Pages 代码不用改，`env.POOL` 照旧可用。

2.3 为什么两件事合在同一个 Worker 里

一个 Worker 完全可以同时导出 DO 类、`scheduled()` 和 `fetch()`，也可以同时声明 `[[migrations]]` 与 `[[triggers]]` `crons` ——没有任何冲突。

合并的三点好处：

1. 少一个部署单元，少配一遍构建变量；
2. 定时作业访问 PoolDO 从「跨 Worker 远程调用」变成本地绑定，少一跳；
3. PoolDO 与调度逻辑同生命周期，不存在「DO 在、调度器没部署上」的错位。

⚠ 注意：`POOL` 绑定在 `engine-worker/` 内部不写 `script_name` ——类和绑定同属一个 script，默认就指向自己。Pages 侧才需要 `script_name` 做远程引用。

2.4 请求流转路径（完整链路）

以一次 `/v1/chat/completions` 流式请求为例：

1. 请求到达 Workers
 - └─ index.ts fetch(): 路径判定
 - /v1/ 前缀 → app.fetch() (Hono)
 - /panel/ 前缀 → env.ASSETS.fetch()
2. 中间件 #1 (最先注册)
 - └─ c.set("wb2aT0", now()) ← 计时起点, performance.now()
 - └─ 建每请求缓存探针 cacheProbe
3. 中间件 #2: 自动迁移
 - └─ needsSchema(path) → ensureSchema(env)
 - KV 版本门: 达标跳过 (1 次 KV 读) / 未达标跑 DDL (batch 压缩)
4. 中间件 #3: 鉴权 (authMiddleware)
 - └─ 解析 Authorization: Bearer
 - └─ 并行发起: [getConfig (KV), loadKeyByHash (D1)] ← 两段互不依赖
 - └─ 计数 auth 段
 - └─ 子密钥: verifyKey (停用/过期/双配额/IP)
 - └─ waitUntil(touchKey) ← 统计写入不阻塞
5. 路由 handler (routes/api.ts)
 - └─ 读请求体 (body 段)
 - └─ timelineOf(c) 建计时表
6. proxyChat()
 - └─ primeConfig + getConfig (cfg 段, SWR 缓存)
 - └─ verifyKeyRequest (realm 归属 + 模型白名单, 需 body 的 model)
 - └─ 并行: [applyPromptPolicy (KV), realModelExists (KV 快照)]
 - └─ Chain() 计算模型候选链
7. 轮转循环 (maxAttempts = min(3 × 链长, 32))
 - └─ poolRPC /internal/pick { acquire: 1 } ← 合并选号+占位+token 刷新
 - └─ DO 内: corePick → 写 lastUsed/usedSeq → freshAuth
 - └─ needsRefresh? → /internal/refresh (已被 freshAuth 覆盖, 通常跳过)
 - └─ chatStream() 上游握手 ← upstream 段到响应头为止
 - └─ 失败: classify → note (惩罚) → release → 轮转/降级/透传
 - └─ 成功:
 - └─ waitUntil(poolRPC /internal/note { kinds:[model_block_clear, success] })
 - └─ streamChat() 逐块转发
 - └─ onEnd → 写日志 + consumeKey + release(cost)
 - └─ request.signal.abort → close(null) + release()
 - └─ 返回前写入计时头
8. 中间件 #1 收尾 (await next() 之后)
 - └─ 写 CORS 头到 c.res.headers ← 必须在 next() 之后
 - └─ 汇总 X-Auth-Cache / X-Models-Cache / X-Cold-Start
 - └─ X-Worker-Uptime

第三部分：目录与代码结构

3.1 完整目录树

```

workbuddy2api-plus-cf/
├── README.md                # 部署说明（命令行版）
├── DEPLOY-NOVICE.md         # 手把手网页教学（30-40 分钟）
├── DEPLOY-WEB.md            # 纯网页部署参数速查
├── AUDIT_AND_ARCHITECTURE.md # ★ Go 源码审计 + 目标架构设计（设计意图最佳入口）
├── package.json
├── tsconfig.json
├── vitest.config.ts
├── wrangler.toml            # ★ 刻意不声明绑定（Dashboard 零配置）
├── worker-configuration.d.ts # Env 绑定类型
├── .gitignore
├──
├── src/                     # 主应用（12,817 行 TS）
│   ├── index.ts             # 入口：路由判定
│   ├── router.ts            # 280 行, Hono 装配 + 中间件链
│   ├── config.ts            # 416 行, 配置 SWR 缓存
│   ├── types.ts             # 243 行, 跨层契约
│   ├── alarms.ts            # PoolDO alarm 调度（兜底路径）
│   ├──
│   ├── routes/              # 路由层（55+ 端点）
│   │   ├── api.ts           (158) # /v1/* + /status + /healthz
│   │   ├── panel.ts         (730) # /panel/api/* 面板数据面
│   │   ├── admin.ts         (200) # /panel/api/accounts|tasks|*_all
│   │   └── login.ts         # OAuth 设备流
│   ├──
│   ├── durable/             # Durable Object
│   │   ├── account-pool.ts (1091) # PoolDO: IO 层 + 状态机
│   │   ├── pool-core.ts    (316) # ★ 选号纯逻辑（无 IO, 可单测）
│   │   ├── session.ts      # 会话粘性
│   │   └── persist.ts       # 状态快照
│   ├──
│   ├── services/            # 服务层（30 文件）
│   │   ├── upstream.ts     (1191) # ★ 上游 HTTP 封装 + 三段超时
│   │   ├── tasks.ts        (902) # 任务中心 22 任务码编排
│   │   ├── proxy.ts        (639) # ★ 反向代理 + 轮转循环
│   │   ├── catalog.ts      (416) # 模型目录
│   │   ├── apikeys.ts      (343) # ★ 子密钥配额与管控
│   │   ├── usage-agg.ts     (335) # 用量聚合（纯函数）
│   │   ├── sse.ts          (329) # ★ SSE 流式/聚合
│   │   ├── desktop.ts      (306) # 桌面端指纹事件
│   │   ├── payload.ts      (287) # 请求体处理
│   │   ├── prompt.ts       (262) # 提示词策略 + 降级门
│   │   ├── credit.ts       (235) # 积分日报
│   │   ├── sanitize.ts     (194) # 指纹脱敏
│   │   ├── efforts.ts      (189) # reasoning effort 能力表
│   │   ├── checkin.ts      (183) # 批量签到（带瞬时重试）
│   │   ├── autoroute.ts    (178) # ★ 模型编排（纯函数）
│   │   ├── health.ts       (168) # 启动自检
│   │   ├── classify.ts      (162) # ★ 错误分类 13 层
│   │   ├── resolveModel.ts (156) # realm 前缀解析
│   │   ├── toolpairing.ts  (155) # tool_call 配对
│   │   ├── timing.ts       (139) # ★ TTFT 分段计时
│   │   ├── growthTasks.ts  (138) # 成长任务
│   │   ├── thinking.ts     (127) # 思维链处理
│   │   ├── buddy.ts        (113) # 猫猫旅行 + 夜猫子
│   │   └── trial.ts        (111) # 加油包

```

```

| | | |—— ids.ts          (109) # ID 生成 (惰性初始化)
| | | |—— cachekey.ts      # ★ 前缀缓存键 (费用优化)
| | | |—— compat.ts        # Responses/Anthropic 兼容层
| | | |—— boot.ts          # ★ 冷启动观测底座
| | | |—— school.ts        # 小程序口径事件
| | | |—— report.ts        # 活跃上报
| | | |—— rates.ts         # 积分倍率
| | | |—— models-snapshot.ts # 模型快照
| | | |—— generated/build-info.ts
| | |
| | |—— storage/          # 存储层 (7 文件)
| | | |—— dl.ts           (486) # D1 封装 + 懒自愈
| | | |—— migrate.ts      (400) # ★ 自动迁移 + 版本门
| | | |—— kv.ts           # KV 封装
| | | |—— r2.ts           # R2 归档
| | | |—— reqlog.ts        # 请求日志
| | | |—— auth.ts          # 凭证解析
| | | |—— usage.ts         # 用量记录
| | |
| |—— engine-worker/      # ② 独立 Worker
| | |—— wrangler.toml      # [[migrations]] + [triggers] crons
| | |—— src/index.ts       (150) # export { PoolD0 } + scheduled()
| | |
| |—— migrations/         # D1 schema
| | |—— 0001_init.sql      # 4 表 5 索引
| | |—— 0002_apikey_quota.sql # 13 个新列
| | |—— 0003_usage_metrics.sql # 复合索引
| | |—— 0004_apikey_prefix.sql # prefix 列
| | |—— 0005_request_logs_usage_cols.sql # 4 个用量列
| | |
| |—— scripts/            # 运维脚本 (1,289 行)
| | |—— wb2api.mjs        (CLI 瘦客户端)
| | |—— preflight.mjs      (部署前体检)
| | |—— smoke.mjs         (部署后冒烟)
| | |—— smoke-panel.mjs   (面板冒烟)
| | |—— fill-ids.mjs      (资源 ID 注入)
| | |—— db-init.mjs       (手工建表兜底)
| | |—— copy-frontend.mjs (前端拷贝)
| | |—— gen-commit.mjs    (构建信息)
| | |—— import-config.mjs (配置导入)
| | |—— import-auths.mjs  (账号导入)
| | |—— deploy-workflow.yml.example
| | |
| |—— vendor/frontend/    # 原面板 (零改动)
| | |—— index.html        (1632)
| | |—— app.js            (3098)
| | |
| |—— test/              # 33 个测试文件 (8,363 行)
| | |—— payload.test.ts   (89 例)
| | |—— catalog.test.ts   (58 例)
| | |—— prompt.test.ts    (54 例)
| | |—— usage.test.ts     (39 例)
| | |—— apikeys.test.ts   (37 例)
| | |—— cli-batch.test.ts  (35 例)
| | |—— pool-core.test.ts  (32 例) ★ 选号逻辑
| | |—— timing.test.ts    (27 例)
| | |—— sse.test.ts       (27 例)

```

```

|—— pool.test.ts          (27 例)
|—— autoroute.test.ts     (26 例)
|—— proxy.test.ts         (23 例)
|—— credits.test.ts       (20 例)
|—— migrate.test.ts       (19 例)
|—— ttft-cache.test.ts    (16 例)
|—— tasks.test.ts         (14 例)
|—— health.test.ts        (11 例)
|—— panel-keys-contract.test.ts (10 例) ★ 前后端契约守卫
|—— panel-auth-split.test.ts  (10 例) ★ 鉴权分离守卫
|—— workers-lifecycle.test.ts (8 例) ★ isolate 生命周期
|—— panel-keys.test.ts      (8 例)
|—— oauth.test.ts          (7 例)
|—— log-retention.test.ts   (7 例)
|—— classify.test.ts        (6 例) ★ 错误分类保序
|—— alarms.test.ts         (6 例)
|—— upstream-timeouts.test.ts (5 例)
|—— panel-logs.test.ts     (5 例)
|—— logs-metrics-reset.test.ts (4 例)
|—— kv-reads.test.ts       (4 例)
|—— frontend-syntax.test.ts  (4 例)
|—— fingerprint.test.ts    (4 例)
|—— panel-usage-cache.test.ts (3 例)

```

3.2 分层职责

层	职责	关键约束
<code>routes/</code>	HTTP 契约、参数解析、响应封装	不写业务逻辑
<code>services/</code>	业务逻辑、上游通信	尽量纯函数（便于单测）
<code>durable/</code>	有状态核心（账号池）	唯一可变状态所在地
<code>storage/</code>	持久化封装	统一 JSON 列编解码

跨层契约集中在 `types.ts`（243 行）：`Realm` / `Auth` / `AccountState` / `ModelCooldown` / `ModelCostEntry` / `CtxVars`。

第四部分：核心实现路径

4.1 账号池 (PoolDO)

4.1.1 架构：纯逻辑与 IO 分离

pool-core.ts (316 行纯函数)		account-pool.ts (1091 行)	
└─ healthy()	健康判定	└─ ensureState()	批量装载
└─ healthyForModel()	模型级健康	└─ getAcct/putAcct	读写
└─ inFlightFull()	在途判定	└─ pickOne()	选号入口
└─ costTier()	成本分层	└─ freshAuth()	token 刷新
└─ floorBlocked()	积分保底	└─ applyNote()	惩罚分发
└─ pickWeighted()	加权抽签	└─ noteError()	熔断
└─ shuffle()	洗牌	└─ noteModelCost()	成本台账
└─ pick()	★ 主入口	└─ fetch()	RPC 路由

为什么这样拆：核心逻辑无 IO 才好单测。 `pool-core.test.ts` 有 32 个用例，全部是纯函数调用，不需要 Miniflare。

4.1.2 状态模型

```
interface AccountState {
  uid, auth, nickname, realm
  status: "healthy" | "cooling" | "disabled"
  disabledReason: string

  // 四层冷却（互不覆盖）
  cooldownUntil + cooldownKind // 账号级冷却
  breakerUntil                // 熔断
  degradeUntil                 // 连败降权
  modelCooldowns: Record<string, ModelCooldown> // 模型级独立冷却

  // 在途
  inFlight: number
  acquiredAt?: number // 超时回收依据

  // 统计
  lastUsed, lastSuccess, usedSeq
  consecutiveFails, consecutiveFailures
  errTotal, retryCount, sessionDeadFails

  // 成本
  credits, creditsTotal
  creditsExpiring, creditsEarliestExpiry, creditsEarliestRemaining
  modelCost: Record<string, ModelCostEntry>
}
```

4.1.3 不可选的判定（四道并列或门）

```
healthy(a, now):
  a.status !== "disabled"
  && a.cooldownUntil <= now
  && (a.breakerUntil ?? 0) <= now
  && (a.degradeUntil ?? 0) <= now
```

关键设计：四个截止是**并列的或门**（不是取最远），任一未到期即不可选——这天然就是「冷却与熔断与降权并存、取更远者不叠加」，不需要显式比较长短。

4.1.4 完整选号流程

输入：候选账号集 `candsIn`, { `realm`, `model`, `exclude`, `now`, `cfg`, `modelRateOf`, `exploreLast` }

Step 1 基础过滤 `baseOk(a)`

- └─ `exclude.has(a.uid)` → 排除（请求级轮换已试过的号）
- └─ `realm && a.realm !== realm` → 排除
- └─ `!healthyOf(a)` → 排除
- └─ `floorBlocked(a, model, cfg)` → 排除（触底号 + 收费模型）

Step 2 首选候选：`baseOk && !inFlightFull`

- └─ 若为空 → 放宽 `inFlight` 限制再试 `baseOk`

Step 3 若仍为空 → 全冷却兜底

- └─ 从软冷却/熔断/降权账号中选 `expiry()` 最早者
- └─ 排除 `disabled` / `hardCooled` / `floorBlocked`
- └─ 返回 { `uid`, `fallback`: true, `fallbackKind`: "breaker"|"soft" }

Step 4 成本分层硬过滤（`model` 非空时）

- └─ 计算所有候选的 `costTier`:
 - | 0 = 已实测免费 (`costPer1k` <= 0)
 - | 1 = 无观测 / 观测过期 (> 6h)
 - | 2 = 已实测收费
- └─ 取 `bestTier` (最小)
- └─ 条件探索: `bestTier==0 && 存在 tier1 && 超窗口`
 - 本次强制切 `tier1-only` (搭车改道)

Step 5 `finishPick()`

- └─ 预计算权重 (`O(n)`)
- └─ 等权洗牌: 候选 > 5 且存在等权 → Fisher-Yates
- └─ 排序: `cost1k` 升 → 权重降 → `uid`
- └─ 截断 top5
- └─ `minPickGap`: 排除 `now - lastUsed < 100ms` 的号
- └─ `eligible` 为空 → LRU 兜底 (全候选取 `usedSeq` 最小)
- └─ 否则 → `pickWeighted` 加权随机抽签

4.1.5 权重计算

```
weightOf(a):
  w = 1.0
  w += (credits / maxCredits) × 10           // 余额比例
  w += lastUsed ? min((now - lastUsed)/3600000 × idle_weight_per_hour, idle_weight_max)
                : idle_weight_max           // 闲置补偿（从未使用 → 满分）

routingWeightOf(a):
  w = weightOf(a)
  if (prefer_expiring && expiringNow(a)) w ×= 3    // 快过期账号 ×3 虚拟实例
```

时钟回拨保护：`idleW < 0 → 0`。

4.1.6 积分保底 (floorBlocked)

```
if (credit_floor <= 0 || !model || credits >= credit_floor) → false (不拦)
```

收费判据两级：

- 1) 本地实测台账 `costPer1k > 0` → 拦
- 2) 无观测时用目录倍率 `rate > 0` → 拦
- 3) 倍率未知 → 放行（保守，避免拦掉内部/别名模型导致号永久失联）

4.1.7 惩罚动作分发 (applyNote)

与 `classify` 的 `note` 字段一一对应：

kind	动作	细节
<code>success</code>	清连续失败计数	—
<code>failures</code>	喂连败降权	仅「不知道原因的失败」
<code>session_dead</code>	连续 3 次才禁用	一次即禁会误杀健康账号
<code>account_fault_11140</code>	立即禁用	需人工重登
<code>account_fault</code> (14017)	软冷却	补完注册可自愈
<code>hard_credit</code> (402)	冷却到 次日 04:00	等 09/21 签到恢复
<code>soft_rate</code>	6004 → 模型级独立冷却 否则按 <code>resetAt</code> 墙钟或指数退避	—
<code>rate_limit_audit</code>	只挂台账，不改变路由	6004 无可解析重置时间
<code>waf_block</code>	账号软冷却 + 抖动	base = 60s
<code>server</code> (5xx)	喂熔断计数（指数退避）	与冷却解耦

kind	动作	细节
<code>model_blocked</code> (11102)	(账号,模型) 负缓存	$6h \times 2^{\min(\text{hits}-1, 4)}$, 封顶 24h
<code>model_block_clear</code>	清 11102 负缓存	不碰 6004 冷却
<code>not_found</code>	固定 60s	不随 <code>soft_rate</code> 退避升级
<code>none</code>	不罚号	内容拦截/参数错/超长

熔断计算 (`noteError`) :

```
consecutiveFailures += 1
if (consecutiveFailures < breaker_threshold) return

d = breaker_cooldown × 1000
for (i = 0; i < retryCount; i++) { d ×= 2; if (d >= max) { d = max; break; } }
consecutiveFailures = 0
retryCount += 1
breakerUntil = now + d
status = "cooling"
```

4.1.8 RPC 协议（跨 Worker 接口）

端点	用途	合并优化
<code>POST /internal/pick</code>	选号	★ 带 <code>acquire:1</code> 合并占位 + <code>freshAuth</code> 合并刷新
<code>POST /internal/acquire</code>	占位（兼容旧版）	—
<code>POST /internal/release</code>	释放在途	★ 带 <code>cost</code> 合并成本台账写入
<code>POST /internal/note</code>	惩罚/成功	★ 带 <code>kinds[]</code> 批量施加
<code>POST /internal/model-cost</code>	成本台账	—
<code>POST /internal/credits</code>	余额回写	—
<code>POST /internal/add</code>	加号	—
<code>POST /internal/remove</code>	删号	—
<code>POST /internal/manage</code>	管理动作	—
<code>POST /internal/refresh</code>	token 刷新	—
<code>GET /internal/status</code>	池状态	—
<code>GET /internal/list</code>	全部账号	—
<code>GET /internal/auth/:uid</code>	取单账号	—

端点	用途	合并优化
POST /internal/scheduler/:task	调度任务	—
POST /internal/alarm / /internal/arm	alarm 控制	—

为什么合并：PoolDO 部署在独立 Worker 上，**每次 RPC 都是一次完整 HTTP 往返**（真机实测 41ms 量级）。原本 `pick → acquire → refresh` 是三次跨 Worker 调用，合并后关键路径只剩一次。

4.1.9 内存态设计（关键陷阱规避）

```
private _accts: Map<string, AccountState> | null = null; // 活对象引用
private _order: string[] | null = null;

private async ensureState(): Promise<Map<string, AccountState>> {
  if (this._accts) return this._accts;
  const [uids, listed] = await Promise.all([
    this.ctx.storage.get<string[]>(INDEX_KEY),
    this.ctx.storage.list<AccountState>({ prefix: "acct:" }), // 1 次读替代 N 次 get
  ]);
  // ... 组装 byUid, index 缺失时补齐
}

private async putAcct(a: AccountState): Promise<void> {
  await this.ctx.storage.put(`acct:${a.uid}`, a);
  if (this._accts) { // 写穿内存
    this._accts.set(a.uid, a);
    if (this._order && !this._order.includes(a.uid)) this._order.push(a.uid);
  }
}
```

⚠ **必须这样做的原因**（源码注释原文）：老实现是「逐 uid storage.get (N+1) + 每次 pick 全量重读」。但直接加缓存会踩一个更隐蔽的坑：**DO 的 storage.get 每次返回独立副本**，acquire/release/note 这些写路径如果读的是副本、而 pick 读的是缓存里的另一个对象，**inFlight /冷却就会各写各的、互相覆盖 —— 比慢更危险**。所以缓存存的必须是「活对象引用」，且**所有写路径一律走 putAcct**（含内存同步），不允许绕过它裸调 `storage.put`。

4.1.10 在途泄漏防护（三重保险）

在途计数 `inFlight` 是**无状态 Workers 下会泄漏的软计数**——客户端断连、Worker 被驱逐、RPC 丢失都会让它只增不减。

保险	实现	触发条件
① 正常回收	<code>onEnd → release()</code>	流正常结束
② 断连兜底	<code>request.signal.abort → release()</code>	客户端中途断开（此时 <code>flush</code> 不触发）
③ 超时强制回收	<code>ACQUIRE_TTL_MS = 10min</code>	pick 时检查 <code>now - acquiredAt > TTL</code>

额外处理： `acquiredAt` 缺失也回收 —— 那是升级前留下的泄漏计数（老版本 `acquire` 不写时间戳），不回收会永远卡死（`no_healthy_account, reasons={in_flight_full:1}`）。

选号时也放宽： `inFlightFull` 全满时不用作硬门槛，否则单账号泄漏一次就永久锁死。

4.1.11 realm 放宽机制

```
const chosen = await this.pickOne(realm, model, exclude, now, cfg);
if (!chosen) {
  const relaxed = await this.pickOne("", model, exclude, now, cfg); // 放宽 realm
  if (!relaxed) → no_healthy_account + diagnose
  ...
}
```

修复的真实 bug： 单域用户——如国际版账号 `realm=global`，但请求模型未带 `global`：前缀导致 `stripRealm` 默认算成 `cn` ——此前会被 `a.realm !== "cn"` 全量过滤，永远 `no_healthy_account`，尽管池里有可用账号。放宽后请求仍按裸模型名出站，选到的账号用自己的 `realm` 落到正确上游域（`basesFor(auth.realm)`），所以不会跨域错打。

4.1.12 诊断能力（diagnose）

`no_healthy_account` 时逐账号列出「为什么不可选」：

```
{
  error: "no_healthy_account",
  realm, model,
  byRealm: { cn: {total, healthy}, global: {...} },
  diagnose: {
    total: 12,
    by_reason: { in_flight_full: 2, cooldown: 5, floor_blocked: 3, ... },
    sample: [{ uid, realize: "pickable_but_unselected" | ... }]
  }
}
```

判读：若 `sample` 里出现 `pickable_but_unselected`，说明兜底逻辑本身有 **bug**（本应可选却没被选上）；若 `total=0`，说明 DO 里根本没加载到账号（绑定/加载问题）。

4.2 错误分类 (classify.ts)

4.2.1 13 层优先级 (顺序即语义)

```
// 0. 11102 模型不存在 (仅 400/404)
if ((status === 400 || status === 404) && isModelBlocked(bodyText))
  → ErrModelBlocked / note: "model_blocked" / rotate: true

// 1. 402 硬积分耗尽
if (status === 402 || code === "upstream_credits_exhausted" || code === "14018")
  → ErrHardCredit / note: "hard_credit" / rotate: true

// 2. 429 软限流 (6004 模型级走模型豁免)
if (status === 429 || code === "rate_limit_exceeded" || code === "6004")
  → ErrSoftRate / note: "soft_rate" / rotate: true

// 3. 12153 会话失效
if (code === "12153" || has("12153") || (status === 401 && has("session")))
  → ErrSessionDead / note: "session_dead" / passthrough: true

// 4. 403 无业务信封 = WAF (IP 级 fail-fast 终止轮转)
if (status === 403 && !e.code)
  → ErrWafBlock / note: "waf_block" / rotate: false

// 5. 404
if (status === 404) → ErrNotFound / note: "not_found" / rotate: true

// 6. 5xx
if (status >= 500) → ErrServer / note: "server" / rotate: true

// 7. 内容拦截 (判在通用 4xx 兜底之前)
if (hasLow("blocked by security policy") || hasLow("unapproved channel") || ...)
  → ErrContentBlocked / note: "none" / rotate: true

// 8. 11101 参数错误
// 9. 11115 提示词过长 → passthrough: true, rotate: false
// 10. 11135 图片非法 → passthrough: true, rotate: false
// 11. 11102 兜底形态 (非 400/404)
// 12. 11140 → Disable / 14017 → 软冷却
// 13. ErrClient 兜底 → note: "failures", 只换号不罚
```

4.2.2 为什么顺序不能错

顺序即优先级：错误码一旦命中立即停止，否则会误罚号。

几个关键排序理由：

- **11102 排最前**：语义最具体（「该后端无此模型」）。但只判 400/404 —— **429 带 11102 属限流语义**，故落到第 2 层。
- **内容拦截排在第 7（通用 4xx 兜底之前）**：落 `ErrClient` 的代价是「换号不罚」，而内容拦截换任何账号都会撞同一审核，轮转纯属浪费。

- **小写副本比对**：文案 marker 一律按小写比对，否则上游返回 "Blocked by security policy" 这类大写形态会漏判。

4.2.3 三个正交标志位

每类错误产出三个正交通道：

标志	含义
note	给账号池的惩罚动作 kind
passthrough	是否透传原文（不轮转）
rotate	是否继续换号

4.3 模型编排 (autoroute.ts)

4.3.1 设计边界

- 只做「选哪个模型」，不做「选哪个账号」（正交，账号由 pool 负责）；
- 编排结果是**候选链**而非覆盖式改写：链首首选，后续仅在可降级错误触发；
- `model_fallback` 递归展开（去重、限深 3、限长 8，**防配置成环**）。

4.3.2 虚拟模型接管

```
IsVirtual(c, model, realExists):
  c.enabled
  && BareOf(model) === (c.virtual_id || "auto")
  && !(realExists && !c.override) // 上游真有同名 → 让位
```

最后一条是关键：**auto** 若在上游真实模型表里存在且未开 **override**，**不接管**——避免静默替换既有能力。

4.3.3 昼夜轮换

```
isDay(c, hour):
  s = day_start (默认 8), e = day_end (默认 23)
  if (s === e) → true // 全天白天
  if (s < e) → hour >= s && hour < e
  else → hour >= s || hour < e // 跨零点窗口

primary(c, hour, wantRealm):
  wantRealm 非空 → 只在该 realm 候选里选 (如 global:auto)
  否则 → isDay ? (day_primary || night_primary) : (night_primary || day_primary)
```

4.3.4 降级触发类别

```
const DEFAULT_KINDS = [  
  "soft_rate", "hard_credit", "model_blocked", "server",  
  "account_fault", "not_found", NO_HEALTHY_ACCOUNT,  
];
```

判据「**换号解决不了、换模型可能解决**」。反之内容拦截 / 参数错 / 上下文超长 / 请求体解析失败**不降级** —— 那是请求本身的问题，换模型也一样撞墙。

4.4 反向代理轮转循环 (proxy.ts)

4.4.1 主循环结构

```

const chain = Chain(autoCfg, rawModel, hourCST(), realExists); // 模型候选链
let ci = 0; // 链索引
let tried: string[] = []; // 本模型已试账号
let perModel = 0; // 本模型换号计数
const maxAttempts = Math.min(MAX_ROTATE * chain.length, MAX_ATTEMPTS_CAP);

for (let i = 0; i < maxAttempts; i++) {
  if (perModel >= MAX_ROTATE && advance()) continue; // 换号额度用尽 → 换模型
  perModel++;

  const { realm, model: bareModel } = stripRealm(chain[ci]);

  // 1. 选号 (合并 acquire + 临期刷新)
  pick = await poolRPC(env, "/internal/pick", "POST",
    { realm, model: bareModel, exclude: tried, stickyKey: sticky, acquire: 1 });

  if (pick.error) {
    // 区分三类故障
    // pool_unavailable → 传输层连不上 (POOL 未绑 / engine 未部署)
    // no_healthy_account → D0 正常响应, 但本模型确实无健康账号
    // 其它 → D0 自身报错
    if (Fallbackable(autoCfg, NO_HEALTHY_ACCOUNT) && advance()) continue;
    break;
  }

  tried.push(uid);

  // 2. 兼容性兜底 (旧版 engine 无 acquire 合并)
  if (pick.acquired !== true) await poolRPC(env, "/internal/acquire", ...);

  // 3. 临期刷新 (已被 freshAuth 覆盖, 通常跳过)
  if (needsRefresh(auth)) { ... }

  // 4. 上游请求
  const up = await chatStream(env, auth, bareModel, work, request.headers);

  if (!up.ok) {
    const c = classify(up.status, txt);
    await note(env, uid, c, bareModel, txt);
    await poolRPC(env, "/internal/release", ...);

    // 内容拦截误报 → 降级提示词重试 (同请求内)
    if (c.kind === "ErrContentBlocked" && (passthrough || append) && !degradedApplied) {
      await triggerDegradate(env);
      work = rewriteSystemPrompt(work, DEGRADED);
      degradedApplied = true;
      tried.splice(tried.lastIndexOf(uid), 1); // 归还重试名额
      continue;
    }

    if (c.passthrough || c.kind === "ErrContentBlocked") return openAIError(...);
    if (Fallbackable(autoCfg, c.kindName) && advance()) continue;
    if (c.rotate && !(await sleep(backoffAfter(i), signal))) break; // 客户端断连
    continue;
  }
}

```

```
// 5. 成功路径
waitUntil(poolRPC(env, "/internal/note", "POST",
  { uid, kinds: ["model_block_clear", "success"], model: bareModel }));
tl.seg.note = 0; // 记 0 而非删除, 让人知道它已不在关键路径

if (body.stream) { /* 流式 */ } else { /* 非流式 */ }
}
```

4.4.2 成功路径的流式处理

```
let rawUsage: string | undefined;
let logged = false;
let released = false;

const close = (usage, note?) => {
  if (logged) return; logged = true;
  waitUntil(log(env, clientIP, userAgent, uid, rawModel, chain[ci], realm,
    "ok", 200, start, usage, rawUsage ?? note));
};

const release = (cost?) => {
  if (released) return; released = true;
  waitUntil(poolRPC(env, "/internal/release", "POST", { uid, ...(cost ? { cost } : {}) }));
};

const res = streamChat(up, request, {
  onUsageRaw: (raw) => { rawUsage = raw; },
  onEnd: (usage) => {
    close(usage, usage ? undefined : "上游流内无 usage 帧");
    if (keyRow) waitUntil(consumeKey(env, keyRow.id, usage?.credit ?? 0, usage?.total_tokens ?? 0));
    release(costOf(usage, bareModel));
  },
});

request.signal.addEventListener("abort", () => {
  close(null, "客户端在流结束前断开");
  release();
});

if (routed) res.headers.set("X-WB2A-Routed-Model", chain[ci]);
// 计时头在返回前写: 此刻 downstream 还没拿到任何字节
```

4.4.3 日志策略的演进（重要）

当前策略：流结束时一次写清（含用量）。

早期是「开局占位 INSERT + 流末 UPDATE 回填」，三个代价：1. 占位 INSERT 是在返回首个字节之前 await 的——一次 D1 往返被压进了 TTFB，而 D1 的往返是毫秒级可变延迟，直接加在用户感知的“首字时间”上；2. 占位 + 回填是**两行写**，D1 免费套餐按写入行数计（10 万行/天），一次对话请求付两行，**等于把日志额度砍半**；3. 少一条 UPDATE 也就少一次「UPDATE 掉进 unprotected 窗口被静默丢弃」的风险。

代价：客户端中途断连、且 Worker 在 abort 回调跑完前就被回收时，这一条没有记录。正常情况 abort 分支会补写一行，真正丢的只有进程级硬中断。

4.5 SSE 流式处理 (sse.ts)

4.5.1 单 TransformStream 设计

```
export function streamChat(upstreamRes, request, opts = {}): Response {
  const body = upstreamRes.body;
  let usage = null, buf = "", seenDone = false;

  const pass = new TransformStream({
    transform(chunk, controller) {
      buf += decode(chunk);
      const { events, rest } = sseSplit(buf);
      buf = rest;
      for (const evt of events) {
        const u = usageOf(evt);
        if (u) {
          if (hasTokens(u)) opts.onUsageRaw?.(JSON.stringify(u));
          usage = pickUsage(usage, u);
        }
        if (/[/\[\]{}]/.test(evt)) seenDone = true;
        controller.enqueue(encode(evt + "\n\n"));
      }
    },
    flush(controller) {
      for (const p of buf.split(/[/\[\]{}]/)) {
        const u = usageOf(p);
        if (u) usage = pickUsage(usage, u);
      }
      const tail = buf.trim();
      if (tail) controller.enqueue(encode(tail.endsWith("\n") ? tail : tail + "\n"));
      if (!seenDone) controller.enqueue(encode("data: [DONE]\n\n"));
      opts.onEnd?.(usage);
    },
  });

  const reader = body.pipeThrough(pass);
  request.signal.addEventListener("abort", () => { body.cancel().catch(() => {}); });
  return new Response(reader, { status: upstreamRes.status || 200, headers: {...} });
}
```

合并理由：原实现是两级管道：tap (decode → 切分 → 每帧 JSON.parse) 再串 sseTransform (再 decode → 再切分 → encode)。同一批字节被解码两遍、切分两遍，多一次 TransformStream 也就多一次队列与一次 enqueue/dequeue 传递。

4.5.2 三个必须知道的坑

坑 1： sseSplit 无限循环 (曾直上生产)

```
export function sseSplit(buf: string): { events: string[]; rest: string } {
  const events: string[] = [];
  for (;;) {
    const norm = buf.replace(/\r\n/g, "\n"); // ★ 每轮基于「最新 buf」重算
    const idx = norm.indexOf("\n\n");
    if (idx < 0) return { events, rest: norm };
    events.push(norm.slice(0, idx));
    buf = norm.slice(idx + 2); // ★ 推进 buf
  }
}
```

旧实现在循环外算一次 `norm` 且循环内不更新，`while` 永远命中第一个 `\n\n`，同一帧被无限 `enqueue`，Worker 内存爆掉被杀，客户端表现为「200 + SSE 头没有任何回复」。独立成纯函数的原因：沙箱 Node 的 `TransformStream` 不稳定，此前透传管线零覆盖，死循环 bug 直上生产。

坑 2: `dataPayloadOf` 必须逐行扫描

```
export function dataPayloadOf(evt: string): string | null {
  const lines = String(evt ?? "").replace(/\r\n/g, "\n").split("\n");
  const payload: string[] = [];
  for (const raw of lines) {
    if (!raw.startsWith("data:")) continue;
    payload.push(raw.slice(5).replace(/^ /, "")); // 只剥一个空格
  }
  if (!payload.length) return null;
  return payload.join("\n").replace(/\s+$/, "") || null; // ★ 只剥尾部，不用 trim()
}
```

⚠ 不能对整个事件串做 `replace(/^data:s?/, "")`：SSE 事件允许多行（`event:` / `id:` / `retry:` / 注释行），而 `^` 只锚定串首——一旦首行不是 data（最常见是上游发 `event: message`），整块 `JSON.parse` 直接失败、usage 被静默丢弃，最终表现为「用量页 token 恒为 0」。⚠ 也不能用 `trim()`：那会把首行 data 内容的前导空格一起吃掉。

坑 3: `pickUsage` 取「最后一个带真实 token 的」

```
function pickUsage(cur, u) {
  if (!u) return cur;
  if (hasTokens(u)) return u; // ★ 优先取带真实 token 的
  return cur ?? u; // 全是空对象时至少留个占位
}
```

上游可能在中途帧先给一个全 0 / 空的 usage（如首帧快照），旧实现 `if (usage) return` 会让首个空对象把真正的末帧 usage 挡在外面 → token 恒为 0。

4.5.3 性能优化三处

优化	实现	收益
字符串快筛	<code>usageOf</code> 先 <code>evt.indexOf('"usage"') < 0</code> 再 <code>JSON.parse</code>	一条流几百到几千帧， 只有末帧带 usage ；原实现对每帧都 <code>JSON.parse</code> ，等于把整条回复的 JSON 又解析一遍
增量聚合	<code>aggregateChat</code> 读一块、切一块、丢一块	原实现峰值内存是「整条响应 × 3」（ <code>text()</code> + <code>replace</code> 复制 + <code>split</code> 子串），Workers 内存上限 128MB
数组拼接	<code>mergeDelta</code> 把片段 push 进 <code>_c / _r / _a / _n</code> ， <code>finalizeMerged</code> 一次 join	避免 <code>str += frag</code> 反复拼接触发多次摊平拷贝

`finalizeMerged` 会删除所有中间态键（`_c / _r / _a / _n`），保证不出现在对外 JSON 里。

4.6 子密钥体系 (apikey.ts)

4.6.1 双鉴权分离

凭据	用途	校验位置
<code>cfg.api_key</code>	调用主钥匙，全权放行 <code>/v1/*</code>	<code>authMiddleware</code>
<code>cfg.admin_key</code>	面板登录口令，只进 <code>/panel/</code>	<code>authMiddleware</code>
<code>wbk_*</code> 子密钥	分发密钥（带配额/IP/白名单）	<code>authMiddleware</code> + <code>proxyChat</code>

```
if (isPanel) {
  // 面板凭据: admin_key 优先; 未单独配置时回退 api_key
  const ok = cfg.admin_key ? timingSafeEqual(token, cfg.admin_key)
    : !(cfg.api_key && timingSafeEqual(token, cfg.api_key));
  ...
}
// 接口凭据: 只认 api_key 与 wbk_ 子密钥, admin_key 在此一律不认
```

这是「登录与调用分离」的实质所在：面板口令泄露也换不来一次模型调用。回退是刻意的——老部署升级后若立刻只认 `admin_key`（默认为空），所有已保存的会话都会 401，管理员等于把自己锁在门外。

4.6.2 状态码刻意分离

情况	状态码	理由
停用 / 过期	403	凭据本身不可用

情况	状态码	理由
双配额用尽	429	避免被读成「密钥无效」
IP / 模型 / realm 不匹配	400	本次请求参数不对，报文透出原因

让客户端能把「配置不对」与「密钥无效」区分开，别一律显示成密钥失效。

4.6.3 两段式鉴权

```
// 第一段（路由层，读 body 之前）
verifyKey(k, clientIP, now):
  enabled === 0          → 403 key_disabled
  expires_at > 0 && now > expires_at → 403 key_expired
  quota > 0 && used_tokens >= quota → 429 quota_exhausted
  quota_credit > 0 && used_credit >= quota_credit → 429 credit_quota_exhausted
  ip_allowlist 有值 && !ipIn      → 400 ip_not_allowed
  max_ips > 0 && ips.length >= max_ips → 400 too_many_ips

// 第二段（proxy 层，读 body 之后）
verifyKeyRequest(k, model, realm):
  realm 限定 + model 为空      → 400 realm_mismatch
  realm 限定 && realm 不匹配    → 400 realm_mismatch (带修复提示)
  模型白名单 + model 为空      → 400 model_not_allowed
  模型白名单 && !modelAllowed  → 400 model_not_allowed
```

拆分原因同 Go：model 要读完 body 才知道，而鉴权发生在读 body 之前。

4.6.4 安全细节

常量时间比较：

```
export function timingSafeEqual(a: string, b: string): boolean {
  if (x.length !== y.length) return false; // 只泄露长度，可接受
  let diff = 0;
  for (let i = 0; i < x.length; i++) diff |= x.charCodeAt(i) ^ y.charCodeAt(i);
  return diff === 0;
}
```

普通 `===` 在第一个不同字节处短路返回，耗时随「猜对的前缀长度」单调上升，理论上可据此逐字节试探。这里比较的是高熵随机串，实际可利用性很低，但成本同样很低——既然要写鉴权分支，顺手把它关掉。

模型白名单精确匹配：

刻意不做前缀/模糊匹配：白名单是「这把钥匙能碰哪些模型」的授权边界，近似匹配等于把边界漏成筛子。

CIDR 匹配（只支持 IPv4，IPv6 按精确字面量）：

```
function cidrMatch(net, ip, bits) {
  let rem = bits;
  for (let i = 0; i < 4 && rem > 0; i++) {
    const take = Math.min(8, rem);
    const mask = take === 0 ? 0 : (0xff << (8 - take)) & 0xff;
    if ((net[i] & mask) !== (ip[i] & mask)) return false;
    rem -= take;
  }
  return true;
}
```

4.6.5 子密钥内存缓存（全项目最值得琢磨的参数取舍）

```
const KEY_CACHE_TTL_MS = 30_000;
const KEY_CACHE_MAX = 1024;
let keyCache = new Map<string, { ts: number; row: ApiKeyRow | null }>();
```

存在理由（真机数据驱动）：

每一次 `/v1` 调用都要为鉴权查一次 D1，实测 `auth` 段在 **123ms 与 247ms 之间反复横跳**——同一把钥匙、同一个操作，能差出一倍。原因是 **D1 库是单区域的，而 Worker 跑在全球各 POP 上，这一跳要跨洋往返**；叠加每个 isolate 首次查询的冷连接，抖动完全不可控。而密钥行的变更频率是「天」量级的，每请求付一次跨洋查询来换取理论上最新的一行数据，是纯粹的浪费。

五处设计细节：

细节	理由
TTL 同时就是配额陈旧窗口	<code>verifyKey</code> 靠行里 <code>used_tokens</code> 判额度，而 <code>consumeKey</code> 在别的 isolate 写 D1。缓存命中时看到的是旧计数，「配额已耗尽」会晚 TTL 才拦住 → 超额放行
刻意不采纳 60~300s	注释明确：这是「愿意多放行多少额度」的 经营决策，不是纯技术参数 。会让一把配额 1000 的钥匙在几分钟内超到什么程度完全不可控
负结果也入缓存	否则任何人拿一个不存在的 <code>wbk_</code> 串反复打网关，就是一条 免费的 D1 放大通道
查失败（异常）不入缓存	「把一次 D1 抖动固化成 60s 的『密钥不存在』会把正常流量误伤成 401。宁可下次再查一次，也不要让网络抖动变成权限判决。」
键取 <code>sha256(token)</code>	缓存里不出现任何可还原凭据的字节

管理面即时失效：

```
export function invalidateKeyCache(hash?: string): void {
  if (!hash) { keyCache.clear(); return; }
  keyCache.delete(hash);
}
```

管理面每次改动密钥都必须调它，否则「停用/改配额/删除」会被内存里的旧行挡住，最长 30s 不生效 —— 面板点了停用、请求却还在放行，这是安全口径，不能只靠 TTL。

4.7 任务中心 (tasks.ts, 902 行)

4.7.1 模块分工

模块	行数	职责
tasks.ts	902	22 个任务码的编排总控
desktop.ts	306	桌面端 5.5.6 行为指纹事件 (14 种事件链)
credit.ts	235	积分日报 (CLI wb2api credit 后端)
checkin.ts	183	批量签到 (带瞬时重试)
growthTasks.ts	138	成长任务 列表/接受/领奖
buddy.ts	113	猫猫旅行 + 连登 + 夜猫子
trial.ts	111	加油包领取
school.ts	85	小程序口径事件上报
report.ts	69	对话活跃上报

4.7.2 核心发现：任务判据靠「客户端指纹」而非端点区分

不是独立端点，而是同一 `POST /v2/report` 上不同客户端指纹。桌面走 `chatBase(copilot.tencent.com)/v2/report` + `extName=workbuddy-desktop`；web 域页面行为走 `www.workbuddy.cn/v2/report`。

三套指纹并存：

指纹	关键头	用于
桌面	<code>X-Domain: chatBase</code> + <code>DESKTOP_UA</code> + <code>X-Product: SaaS</code>	RichMeow_Chat、模板/专家/设计画布
小程序	<code>X-Client-Product: workbuddy-mp</code> <code>X-Client-Platform: mp-weixin</code> <code>X-Platform: wechatmp</code>	Sequential_Tasks_*、school_season

指纹	关键头	用于
Web	WEB_UA	页面行为事件

4.7.3 任务码表

```
const mpTaskCodes = new Set([
  "school_season",
  "Sequential_Tasks_1" ... "Sequential_Tasks_7",
]);
```

4.7.4 反作弊对抗（实战价值最高的部分）

机制	实现常量	原因
真人节奏	MP_CHAT_EVENT_GAP_MS = 45_000	上游对 Sequential_Tasks_3 「5 次有效对话」有反作弊：数秒级连发先计数后被判定无效整体回滚。实测 45s 间隔全存活
accept 回读验证	重试 2 次 + MP_ACTION_GAP = 2000	上游存在「200 + OK 但 accept 未真正登记」的形态，此时上报事件不归账，任务永远点不亮
异步计分轮询	CLAIM_POLL_ATTEMPTS = 4 / CLAIM_POLL_GAP = 3000	上游计分延迟数秒才刷新，一次回读会误判「未达标」
连登告警信号	GrowthStreak 返回 0	即「上报 200 但静默丢弃」的信号

4.7.5 夜猫子任务

```
export async function RunNightChats(auth, env, need, reportModel): Promise<number> {
  let ok = 0;
  for (let i = 0; i < need; i++) {
    const res = await chatStream(env, auth, "glm-5.2",
      { messages: [{ role: "user", content: "1+1等于几? 直接回答。" }], stream: true },
      new Headers());
    if (!res.ok) throw new Error(`第 ${i + 1} 次对话失败: http=${res.status} ...`);

    // 读干 SSE 流 (上限 1MB, 避免残留连接)
    const reader = res.body?.getReader();
    if (reader) {
      let read = 0;
      for (;;) {
        const { value, done } = await reader.read();
        if (done) break;
        read += value?.length ?? 0;
        if (read > (1 << 20)) break;
      }
    }
    await reportModel(auth, env, `wb2api-night-${Date.now()}-${i}`, "", "glm-5.2", "GLM-5.2");
    ok++;
    await sleep(4000);
  }
  return ok;
}
```

夜猫子窗口: `InNightWindow()` → `h >= 23 || h < 8`。

一处很典型的取舍: 任务判据要求「真实对话」, 那就**真跑一次对话** —— 内容是「1+1等于几?」极短, **消耗可忽略**, 但省掉了逆向事件 schema 的风险。

4.7.6 幂等性

全部动作幂等: 已 claimed / 已达标直接跳过, 不重复消耗上游配额。

这是必要的 —— `runScheduledJobs` 每小时触发, 且外部 cron 与 DO alarm 两条驱动路径互为兜底, **同一作业可能被触发多次**。

4.8 上游通信 (upstream.ts, 1191 行)

4.8.1 双 realm 端点表

```
const CN = {
  chat: "https://copilot.tencent.com",
  billing: "https://www.codebuddy.cn",
  web: "https://www.workbuddy.cn",
};
const GLOBAL = {
  chat: "https://www.workbuddy.ai",
  billing: "https://www.workbuddy.ai",
  web: "https://www.workbuddy.ai",
};
```

4.8.2 上游端点全集

/v2/chat/completions	对话 SSE
/v2/plugin/auth/token/refresh	token 刷新
/v2/plugin/auth/keepalive	保活
/v2/enterprises/personal/models	模型目录
/v3/config	配置 (与上者并发合并)
/v2/billing/meter/get-user-resource	余额/积分包
/v2/billing/meter/daily-checkin	签到
/v2/activity/growth/tasks[/accept]	任务中心
/v2/report	事件上报 (三套指纹共用)
/v2/user-asset/appearance/set	主题设置
/activity/growth/buddy/travel/{status,depart,claim}	猫猫旅行
/activity/growth/buddy/{first,agreement}	领养
/activity/growth/streak	连登

4.8.3 出站头构造

```
export function buildHeaders(auth, env, extra?): Headers {
  h.set("Authorization", "Bearer " + auth.accessToken);
  h.set("User-Agent", cfg.upstream.user_agent);
  h.set("Content-Type", "application/json");
  h.set("Accept", "application/json, text/event-stream");
  if (auth.device_token) h.set("X-Device-Token", auth.device_token);
  if (cfg.upstream.client_name) {
    h.set("X-IDE-Name", "workbuddy2api");
    h.set("X-IDE-Client-Name", cfg.upstream.client_name);
    h.set("X-IDE-Client-Version", cfg.upstream.client_version);
    h.set("X-IDE-Version", cfg.upstream.client_version);
  }
  h.set("X-Conversation-Request-ID", crypto.randomUUID());
  // ★ 稳定设备指纹三头 (纯 uid 哈希)
  for (const [k, v] of Object.entries(deviceHeaders(auth.uid))) h.set(k, v);
  ...
}
```

设备指纹三头的注释很关键：纯 uid 哈希、不依赖桌面端/SDK，任何环境都带 —— 原版在云端/纯 Linux 跑通余额正是靠它，不能丢。

4.8.4 流式三段超时（关键修复）

```
export async function withStreamTimeouts(req, opt: { totalMs, headerMs, idleMs }): Promise<Response> {
  const ctrl = new AbortController();
  let h, i, t;
  if (opt.totalMs > 0) t = setTimeout(() => ctrl.abort(), opt.totalMs);
  if (opt.headerMs > 0) h = setTimeout(() => ctrl.abort(), opt.headerMs);
  // idleMs 每收到一个 chunk 就重置
  ...
}
```

为什么不能只用 `withTimeout`：`fetch()` 在响应头到达时就 resolve，原实现紧随其后的 `finally { clearTimeout }` 会把总超时一起清掉——于是 **120s** 的总超时实际只覆盖到首字节，流阶段完全裸奔。上游若半途卡住不发也不关（网关挂起、连接被中间设备静默丢弃），这个请求会一直挂着：用户侧看到「转圈不结束」，Worker 侧持续占着一个在途名额与一个出站连接，还会把 10ms CPU/请求的额度慢慢吃掉。

段	配置项	语义
<code>headerMs</code>	<code>upstream.header_timeout_seconds</code>	首字节超时 —— 头都回不来多半是上游不可用，早失败让轮转试下一个号
<code>idleMs</code>	<code>upstream.idle_timeout_seconds</code>	流间空闲超时 —— 每收一个 chunk 就重置， 不影响正常长回复
<code>totalMs</code>	<code>upstream.timeout_seconds</code>	总时长兜底

4.8.5 前缀缓存键（费用优化）

```
export async function buildCacheKey(uid: string, conversation: string): Promise<string> {
  const uid8 = uid ? uid.slice(0, 8) : "-";
  const data = new TextEncoder().encode(uid + "|" + conversation);
  const buf = await crypto.subtle.digest("SHA-256", data);
  const hex = [...new Uint8Array(buf).slice(0, 16)].map(b => b.toString(16).padStart(2, "0")).join("");
  return "wb2a-" + (uid8 || "-") + "-" + hex;
}
```

逆向实测数据（注释原文）：

上游服务端支持 `prompt_cache_key`，同一段 8k token 前缀：- 不带 → `prompt_cache_hit_tokens=0`，`credit≈0.34` - 带 → `prompt_cache_hit_tokens=7808`，`credit≈0.02`（费用降约 17×）

配套的字段盲区修复：

上游（`/v2/chat/completions`）实测返回的是 `prompt_cache_hit_tokens` —— 此前本文件只认 OpenAI 口径的 `prompt_tokens_details.cached_tokens` / `cache_read_input_tokens`，结果上游唯一真在返回的字段被整个忽略，命中率维度恒为 0。

`resolveConversationID` 只认 `conversationId`，绝不回落 `user_id` —— 会污染按对话聚合的判据。

第五部分：可观测性体系

这套自研观测是全项目最有价值的部分，思路可移植到任何 Workers 项目。

5.1 分段计时 (timing.ts)

5.1.1 十段划分

```
SEG_ORDER = ["migrate", "auth", "body", "cfg", "prompt", "models", "pick", "refresh", "note", "upstream"]
```

按关键路径的先后固定排序，而非对象插入顺序：

对象插入顺序取决于代码执行路径（错误路径会跳过若干段），直接遍历会让每次响应的字段顺序都不同，**diff 起来很痛苦**。

段	含义
migrate	D1 自动迁移（版本命中时 ≈ 0 ）
auth	子密钥的 D1 查询（管理员钥匙走内存比对，不输出此段）
body	读取请求体（长上下文时变大）
cfg	读网关配置（KV，SWR 缓存）
prompt	提示词策略（KV：降级门状态）
models	模型存在性判定（60s 快照）
pick	选号 RPC（跨 Worker 往返，轮转时累加）
refresh	token 刷新（已被 freshAuth 覆盖，通常缺席）
note	成功后清负缓存 + 记成功（移到 waitUntil 后记为 0）
upstream	上游握手到响应头为止

5.1.2 total 的定义

```
const total = Math.max(0, now() - tl.t0);
```

total 恒 = 生成时刻 - t0，**不是**各段之和：关键路径外还有 Workers 调度、CORS 中间件、路由匹配等未计入段的开销，**二者之差本身就是有用信息**（差值大 = 计时漏段了）。

5.1.3 单调时钟（关键约束）

```
export function now(): number {
  return perf ? perf.now() : Date.now();
}
```

两条必须遵守的约束：

1. 必须用 `performance.now()`： `Date.now()` 会被 NTP 校正回拨，计时差值一旦为负，Server-Timing 的解析方会直接丢弃整条头。

真机上 `body` 段恒显示为 0 就是这个坑：几十 KB 请求体的读取加 `JSON.parse` 大约 0.3~0.8ms，`Date.now()` 的毫秒分辨率把它整个抹成 0，于是看不出「到底是快还是压根没设计时」这种本质区别。

2. 全链路只能用这一支表：`performance.now()` 返回「isolate 启动至今」（几百~几十万量级），`Date.now()` 是 epoch（1.7e12 量级），两者不能相减。任何一个埋点图省事写了 `Date.now()`，那一段的差值就会变成天文数字（或负到被归零），整条归因数据当场报废。

5.1.4 输出格式

```
Server-Timing: migrate;dur=0, auth;dur=12, body;dur=4, cfg;dur=0.3, pick;dur=48, note;dur=0, upstream;dur=310,
total;dur=420
X-WB2A-Timing: migrate=0 auth=12 body=4 cfg=0.3 pick=48 note=0 upstream=310 total=420
```

双格式理由：

为什么用 `Server-Timing` 而不只自定义一个头：它是 HTTP 标准，Chrome DevTools 的 Timing 面板、Safari、`curl -w '%header{server-timing}'` 都原生识别，不必接工具链。额外再给一份紧凑纯文本的 `X-WB2A-Timing`，是为了在终端里贴出来对比时一眼能读完（`Server-Timing` 的 `;dur=` 语法很啰嗦）。

精度：`round1()` 保留一位小数。原先用 `Math.round` 取整，亚毫秒段（`body` / `cfg` / `note` 这类已优化到 <1ms 的）会一整片糊成 0，看不出优化到底生效没有。

空表返回空对象：

没有记到任何段时不要输出一个只有 `total;dur=0` 的噪声头——那会让人误以为计时生效了但其实什么都没记。

5.1.5 分段累加 vs 覆盖

```
export function markSince(tl, name, since): number {
  const ms = Math.max(0, now() - since);
  tl.seg[name] = ms;          // 覆盖
  return ms;
}
export function addElapsed(tl, name, ms): void {
  tl.seg[name] = Math.max(0, (tl.seg[name] ?? 0) + ms); // 累加
}
```

pick 用累加 —— 轮转重试会打好几轮，用户关心的是「选号这件事总共花了多久」。

5.2 冷启动观测 (boot.ts)

5.2.1 四类诊断头

头	含义
X-Worker-Uptime	本 isolate 已存活毫秒 (<code>Date.now()</code> - <code>WORKER_START</code>)
X-Cold-Start	是否发生过缓存 miss (1 = 本次请求关键路径至少付了一次后端往返)
X-Auth-Cache	配置 + 子密钥的缓存命中情况 (hit / miss)
X-Models-Cache	模型目录快照的缓存命中情况

5.2.2 设计核心：把两种冷启动成因分开

「isolate 刚起来」 → 要改的是：砍启动期 I/O
「cache TTL 到期」 → 要改的是：拉长 TTL

二者要改的东西完全不同，混在一起 p95 没有统计意义。

5.2.3 X-Cold-Start 的语义重写

```
h.set("X-Cold-Start", authHit === "miss" || modelsHit === "miss" ? "1" : "0");
```

重写为「是否发生过缓存 miss」：这比「isolate 首请求」更有用 —— 后者和缓存实际命中无关。

5.2.4 模块级 vs 请求级的划分

```
const WORKER_START = Date.now(); // 模块级：本 isolate 只记一次
export function newCacheProbe(): CacheProbe { // 请求级：每请求一份
  return { config: [], key: [], models: [] };
}
```

一部分是 **isolate** 的属性（不是请求的属性），而 Hono 的 Context 是每请求一份、且只能在 `app.fetch` 内部拿到。那些「本 isolate 只记一次 / 跨请求共享」的量必须放在模块作用域。

WORKER_START 刻意用 `Date.now()`：uptime 是「差值」语义，基准点只取一次，不受两种时钟不可相减的影响；且这是响应头而不是计时段，不参与任何 `now()` -based 的相减。

5.3 联动读法（实战判读）

现象	推断
<code>pick + note</code> 大	跨 Worker RPC 是瓶颈
<code>upstream</code> 大	上游握手/排队慢，本地无事可做
<code>auth</code> 大 + <code>X-Auth-Cache: miss</code>	D1 跨洋查询，缓存未命中
<code>auth</code> 大 + <code>X-Auth-Cache: hit</code>	抖动来自其它环节（需查 config SWR）
<code>migrate</code> 大	冷启动真付了迁移成本
<code>total</code> 远大于各段之和	计时漏段了
<code>body</code> 大	客户端上传慢 / 长上下文

验证命令：

```
curl -sD - -o /dev/null -X POST https://<域名>/v1/chat/completions \
  -H "Authorization: Bearer <KEY>" -H "Content-Type: application/json" \
  -d '{"model": "cn:hy3", "messages": [{"role": "user", "content": "hi"}]}' \
  | grep -i 'x-wb2a-timing\|server-timing'
```

5.4 CORS 与 Expose-Headers

```
h.set("Access-Control-Expose-Headers",
  "Server-Timing, X-WB2A-Timing, X-WB2A-Routed-Model, X-Cold-Start, X-Worker-Uptime, X-Auth-Cache, X-Models-Cache")
;
```

Expose-Headers 是自定义响应头能否被浏览器 JS 读到的开关：同源不看它，但跨域客户端要认 **X-WB2A-*** 就必须列出，否则 `headers.get()` 恒为 `null` —— 头在 HTTP 层面明明存在，抓包看得到、代码读不到。

第六部分：Go → Cloudflare 迁移映射

6.1 状态外移总表

原 Go 形态	落点
全局 <code>map[uid]*Auth</code> + 锁	单实例 Durable Object <code>PoolDO</code>
<code>auths/*.json</code> / <code>state.json</code>	PoolDO Storage
<code>usage.json</code> / <code>request-logs/*.jsonl</code>	D1 (查询) + R2 (归档)
<code>keys.json</code>	D1
<code>model.json</code> / <code>output_probes.json</code>	KV (TTL 缓存)
<code>scheduler.Run</code> 常驻 goroutine	engine Worker 的 Cron Triggers
Upstash/Redis	已移除 (纯 DO Storage)
<code>os.Getenv</code> / <code>config.json</code>	Pages Secrets (敏感) + KV (非敏感)
<code>fs</code> / <code>path</code> / <code>child_process</code>	不存在 → 全部外移

6.2 依赖替代表

Go 依赖	用途	TS 替代
<code>net/http</code> , <code>ServeMux</code>	路由	Hono
<code>encoding/json</code>	序列化	原生 <code>JSON</code> + <code>zod</code>
<code>regexp</code>	错误分类	原生 <code>RegExp</code>
<code>sync</code> , <code>atomic.Bool</code> , <code>sync.Map</code>	并发/状态	Durable Object (顺序单线程, 天然替代锁)
<code>time</code> + <code>FixedZone("UTC+8")</code>	墙钟/时区	原生 <code>Date</code> (固定 UTC+8, 勿依赖 tzdata)
<code>go-redis/v9</code>	Upstash 镜像	移除 (DO Storage 已天然共享状态)
<code>go.uber.org/atomic</code>	原子布尔	不需要 (DO 单线程)
<code>cespare/xxhash</code> , <code>go-rendezvous</code>	redis 内部	不需要
<code>crypto/rand</code>	api_key 生成	Web Crypto <code>crypto.getRandomValues</code>

6.3 模块映射表（部分关键项）

Go 文件 / 函数	TS 模块	云资源
<code>cmd/server/main.go</code> , <code>config.go</code>	<code>src/config.ts</code>	KV + Secrets
<code>internal/server/handler.go</code>	<code>src/routes/api.ts</code>	Hono + PoolDO
<code>internal/server/compat_*.go</code>	<code>src/services/compat.ts</code>	—
<code>internal/panel/panel.go</code>	<code>src/routes/panel.ts</code>	Hono + D1 + PoolDO
<code>internal/panel/index.html</code> / <code>app.js</code>	<code>dist/panel/*</code> （原样拷贝）	Pages 静态
<code>internal/panel/autotask.go</code> , <code>taskcenter.go</code>	<code>src/services/tasks.ts</code>	PoolDO + D1
<code>internal/auth/auth.go</code>	<code>src/storage/auth.ts</code>	PoolDO Storage
<code>internal/upstream/client.go</code>	<code>src/services/upstream.ts</code>	fetch
<code>internal/upstream/sse.go</code>	<code>src/services/sse.ts</code>	ReadableStream
<code>internal/upstream/classify.go</code>	<code>src/services/classify.ts</code>	—
<code>internal/pool/*</code>	<code>src/durable/account-pool.ts</code> + <code>pool-core.ts</code>	PoolDO Storage
<code>internal/pool/persist.go</code>	<code>src/durable/persist.ts</code>	PoolDO Storage
<code>internal/scheduler/*</code>	<code>src/alarms.ts</code> + engine cron	Cron Triggers
<code>internal/session/*</code>	<code>src/durable/session.ts</code>	PoolDO Storage
<code>internal/redisstore/*</code>	移除	—
<code>internal/usage/*</code>	<code>src/services/usage-agg.ts</code> + <code>storage/usage.ts</code>	D1
<code>internal/reqlog/*</code>	<code>src/storage/reqlog.ts</code> + <code>d1.ts</code>	D1 + R2
<code>internal/apikeys/*</code>	<code>src/services/apikeys.ts</code>	D1
<code>cmd/{credit,login,signin,trial}</code>	<code>scripts/wb2api.mjs</code> （瘦客户端）	—

6.4 前端零改动的保障机制

三条一致性约束：

1. 路径完全一致： `/v1/*` 、 `/panel/*` 原样保留
2. 鉴权一致：面板与 API 均 `Authorization: Bearer <key>`

3. 字段名逐字保留：`/status`、`/panel/api/overview`、`/v1/models` 的字段名逐字保留（前端靠这些渲染）

机制保障：`test/panel-keys-contract.test.ts`（10 例）做契约守卫。

错误信封统一：

```
{ "error": { "message": "...", "type": "api_error", "code": "...", "gateway_hint?": "..." } }
```

错误 `code` 名（如 `content_blocked`、`prompt_too_long`、`upstream_credits_exhausted`）逐字保留。

6.5 CLI 的云原生重构

为什么不能逐字移植：Go 的 4 个二进制全部依赖「遍历本地 `auths/workbuddy-*.json` 目录」：

Go 依赖	Workers 下的现实
<code>filepath.Glob("./auths/*.json")</code>	账号池在 PoolDO 的存储里，没有目录
<code>a.SaveAtomic()</code> 写回 token	凭证在 DO Storage，由 DO 持久化
<code>os.Getenv("WB2A_AUTH_DIR")</code>	无本地环境
<code>time.Sleep(200ms)</code> 逐个串行	应走并发池
<code>up.New()</code> 每进程独立客户端	全网关共享一个 PoolDO

强行保留"遍历目录"只会得到一个永远空转的脚本。

方案：能力上移 + 瘦客户端

Go CLI	服务端能力	HTTP 端点	CLI 子命令
<code>cmd/credit</code>	<code>services/credit.ts</code>	<code>GET /panel/api/credits?pretty=1</code>	<code>wb2api credit</code>
<code>cmd/signin</code>	<code>services/checkin.ts</code>	<code>POST /panel/api/signin_report</code>	<code>wb2api signin</code>
<code>cmd/trial</code>	<code>services/trial.ts</code>	<code>POST /panel/api/trial</code>	<code>wb2api trial</code>
<code>cmd/login</code>	<code>routes/login.ts</code>	<code>POST /panel/api/login/start + poll</code>	<code>wb2api login / poll</code>

三个刻意保留的行为差异：

1. `signin_report` 是同步的，`checkin_all` 不是

后者是定时任务入口（fire-and-forget）；前者是 CLI 入口，核心价值就是那张逐账号表格（谁 OK / 谁已签 / 谁 AUTH_INVALID），fire-and-forget 会把这个价值丢掉。并发压到 3（同 `/packages`），否则撞 Workers 请求时限。

2. `TotalDosage` 下限只在日报口径生效

Go `cmd/credit` 用上游 `TotalDosage` 作 `size` 下限；`UserResourceDetailedWithExpiry` 不做这步。
`applyDosageFloor` 只在 `credit.ts` 里用，不动 `getCreditsDetailed` —— 后者是成本台账/选号权重的唯一数据源，随意改口径会让三者全部失真。

3. `checkin.ts` 走瞬时重试，`upstream.dailyCheckin` 不走

Go 侧 `DailyCheckin` 包在 `retryBillingTransient` 里（签到后偶发 500 会让账号整天漏签）。
`upstream.dailyCheckin` 是裸调用，只被 OAuth 新号入池那条路径用（失败不影响任何既有状态，重试无意义）。

第七部分：部署路径

7.1 三条部署路线

路线	文档	适用
命令行版	<code>README.md</code> 第 4 节	有 wrangler CLI 环境
手把手网页版	<code>DEPLOY-NOVICE.md</code>	没有本地电脑，30-40 分钟
参数速查	<code>DEPLOY-WEB.md</code>	有经验只要参数

7.2 零配置部署的核心技巧

`wrangler.toml` 刻意不写 `pages_build_output_dir`、也不写任何绑定：

Cloudflare 的规则是——Pages 项目一旦有 `wrangler.toml` 且被识别为生产配置，Dashboard 里对应的字段就变成**只读、点不动**，绑定只能靠改文件填 ID。留空之后 5 个绑定全在 Dashboard 的 Settings → Functions 里点。

页面侧需在 Dashboard 配 5 个绑定：

```

KV namespace bindings → WB2A_CONFIG (非敏感配置)
KV namespace bindings → WB2A_CACHE (模型目录缓存)
D1 database bindings → WB2A_DB (用量/请求日志/子密钥)
R2 bucket bindings → WB2A_LOGS (请求归档，可选)
Durable Object bindings → POOL (指向 workbuddy2api-engine)
```

△ Production 与 Preview 两套环境**各配一遍**；改完要 Retry deployment。△ 变量名必须**一字不差**。

7.3 部署顺序（不可颠倒）

```

Step 1 创建资源
wrangler kv namespace create WB2A_CONFIG
wrangler kv namespace create WB2A_CACHE
wrangler d1 create workbuddy2api
wrangler r2 bucket create workbuddy2api-logs    # 可选

Step 2 部署 Worker ②（必须早于 Pages）
npm run deploy:engine
# 或: npx wrangler deploy --config engine-worker/wrangler.toml

Step 3 设 Secrets（按 Worker 独立存储）
npx wrangler secret put WB2A_API_KEY --config engine-worker/wrangler.toml
wrangler pages secret put WB2A_API_KEY --project-name workbuddy2api-pages

△ Secret 是按 Worker 独立存储的，Pages 项目设的那些不会自动带过来

Step 4 构建与部署 Pages
npm run build          # esbuild → dist/_worker.js + 拷贝前端
wrangler pages deploy dist

Step 5 验证
npm run preflight
WB2A_URL=... WB2A_API_KEY=... npm run smoke
  
```

7.4 D1 自动迁移（无需手工步骤）

已自动化：Worker 在首个请求时自检并建表（`src/storage/migrate.ts`），部署后无需任何手工步骤。空库 → 首次访问 `/healthz` 即完成 **4 表 5 索引 19 列** 的搭建。

7.4.1 版本门机制（冷启动优化核心）

问题：迁移原先跑在每个 API 请求、Hono 之前，且是纯串行的。即便数据库早已是最新 schema，它仍会老实走完：

8 条 CREATE + 2 条 PRAGMA + 3 条索引 = 14 次串行 D1 往返

而 promise 缓存是**模块级**的，isolate 一回收就重新付一遍。

两层优化：

层	机制	效果
① 版本门	迁移成功后把版本号写 KV；冷启动先花 1 次 KV 读 （亚毫秒）确认达标 → 完全跳过 DDL	省掉 14 次 D1 往返
② batch 压缩	<code>d1.batch()</code> 打包：9 条建表索引一批、3 条探测一批、缺列一批	真需迁移时 14 → 3~4 次

```

const SCHEMA_TARGET = 5;

async function guarded(env: Env): Promise<MigrateState> {
  const d1 = env.WB2A_DB;
  if (!d1) return { status: "skipped", reason: "WB2A_DB 未绑定" };

  if (!bypassGate) {
    const v = await readSchemaVersion(env);
    if (v !== null && v >= SCHEMA_TARGET) {
      return { status: "ok", created: [], added_cols: [], via: "version_gate" };
    }
  }
  bypassGate = false;

  const r = await migrate(env);
  if (r.status === "ok") await writeSchemaVersion(env, SCHEMA_TARGET); // 只有真成功才写
  return r;
}

```

7.4.2 失效即退化原则

版本门失效即退化，绝不成为新的故障源：KV 未绑定、读失败、值非法，一律当作「版本未知」照跑迁移。KV 是最终一致的，但这里只存一个单调递增的版本号——读到旧值最坏情况是多跑一次幂等迁移，**不会漏迁移**。

7.4.3 三条硬约束

约束	原因
一条语句一次 <code>prepare()</code>	D1 不支持多语句批处理
加列前先 <code>PRAGMA table_info</code> 探测	SQLite 的 <code>ADD COLUMN</code> 没有 <code>IF NOT EXISTS</code>
吞掉 <code>duplicate column name</code>	多个 isolate 并发首触发时的正常竞态

⚠ 改 `migrations/*.sql` 时必须同步 `src/storage/migrate.ts`（`preflight.mjs` 有静态一致性检查）。

7.4.4 `bypassGate` 的测试陷阱

```

export function resetSchemaCache(): void { inflight = null; } // 不穿透版本门
export function forceSchemaMigration(): void { inflight = null; bypassGate = true; }

```

⚠ `bypassGate` 只由 `forceSchemaMigration()` 置起，不要挂到 `resetSchemaCache()` 上。后者是「清空模块级缓存」的通用入口，被测试的 `beforeEach` 普遍调用——若它也置起这个标志，版本门在测试里就永远不会被走到，等于没写。

7.5 部署校验三件套

7.5.1 preflight.mjs (部署前)

存在理由：这三个阻塞项的共同特点是**部署命令照样成功**，问题在几小时后才暴露：1. `wrangler.toml` 的占位符 ID 没换 → 绑到错误/不存在的资源；2. D1 没建表 → 部署成功，首个写请求才 500；3. `WB2A_API_KEY` Secret 没设 → 部署成功，**面板全锁（鉴权恒 401）**。逐条 `wrangler pages deploy` **不会拦你任何一条**。

检查项：1. Pages 绑定配置方式（Dashboard 零配置 / 文件声明）2. `migrations/*.sql` 与 `migrate.ts` 一致性（静态检查）3. 构建产物 4. `--remote`：远端表清单 + R2 桶存在性 + 线上健康

7.5.2 smoke.mjs (部署后)

打真实端点，验证「能连上、能鉴权、核心读写通」。

7.5.3 /healthz 两层语义（关键设计）

状态	HTTP	含义	该不该报警
<code>ready: false</code>	503	配置坏了（D1 缺表 / 密钥为空 / 绑定不可用）	☒ 该报警
<code>ready: true, healthy: false</code>	200	配置没问题，只是池里还没可服务账号	☒ 新部署的正常中间态，不该报警

`checks` 逐项给出原因与修复命令。

7.6 fill-ids.mjs (资源 ID 注入)

核心洞察：

Pages 项目的 `wrangler.toml` 是配置的唯一真源 —— 只要它在，Dashboard 里的绑定就是只读、点不动的。所以资源 ID 没法靠网页点击填，只能写进文件。

解法：构建机在 `checkout` 之后、部署之前跑脚本，用构建环境变量替换 `REPLACE_WITH_*` 占位符。**仓库里的 `wrangler.toml` 永远保持占位符原样，ID 不进版本库。**

```
CF_KV_CONFIG_ID=xxx CF_KV_CACHE_ID=yyy CF_D1_ID=zzz npm run fill:ids
# --lenient 缺哪个跳过哪个；--dry-run 只打印不落盘
```

R2 的优雅降级：

```
const R2_ENABLED = R2_BUCKET.length > 0;
// 没配 → disableSection(text, "[[r2_buckets]]") 逐行注释掉，保留行序便于人工核对
```

R2 桶不存在时 `wrangler deploy` 会硬失败（`R2 bucket 'xxx' not found [code: 10085]`），而建桶通常要绑支付方式。所以默认不绑——代价是日志归档关闭，其余功能照常（`archiveLogs` / `listLogDays` 都对未绑定做了降级）。

兜底自检：写完后重新扫一遍，若仍有 `REPLACE_WITH_*` 残留就 `exit(1)`。

7.7 两条定时驱动路径

方式 A（推荐）：engine Worker 原生 cron

cron (UTC)	作用
<code>0 * * * *</code>	每整点：按北京时间判断该跑哪些作业（ <code>runScheduledJobs</code> ）
<code>30 17 * * *</code>	UTC 17:30（北京时间 01:30 低峰）：7 天前日志 D1 → R2 归档

方式 B（兜底）：DO alarm 自调度

```
curl -X POST https://<pages.dev>/panel/api/scheduler/arm -H "Authorization: Bearer $KEY"
```

`PoolDO` 会在每个整点执行对应任务并 `setAlarm(nextHour)` 自调度。

方式 C（最稳，零 alarm 复杂度）：外部 cron 调面板接口

```
curl -X POST https://<pages.dev>/panel/api/checkin_all -H "Authorization: Bearer $KEY"
curl -X POST https://<pages.dev>/panel/api/travel_all -H "Authorization: Bearer $KEY"
curl -X POST https://<pages.dev>/panel/api/activity_all -H "Authorization: Bearer $KEY"
curl -X POST https://<pages.dev>/panel/api/keepalive_all -H "Authorization: Bearer $KEY"
```

三者复用同一套调度逻辑，互为兜底。

7.8 定时作业时刻表

任务	默认整点（北京时间）	接口触发
签到 + 余额解冻	9, 21	<code>/checkin_all</code>
猫猫旅行（领养/派出/领奖）	9, 21	<code>/travel_all</code>
活跃上报（连登+解锁）	10	<code>/activity_all</code>
token 保活	22	<code>/keepalive_all</code>
夜猫子（glm-5.2 对话补足）	23	—
成长任务队列	1	<code>/tasks/run_queue</code>

任务	默认整点（北京时间）	接口触发
余额后台刷新	每 5min	<code>/balance_all</code>

7.9 已知环境限制

限制	应对
Pages 无 Cron Triggers	engine Worker cron（主）+ DO alarm（备）+ 外部 cron（备）
Pages 无 Queue 消费者	任务中心用 PoolDO 同步/半异步（ <code>ctx.waitUntil</code> ）
无文件系统	全部外移到 KV / D1 / R2 / DO Storage
无全局可变内存	单实例 DO 顺序单线程替代锁
长任务 CPU 时间	全量任务 <code>waitUntil</code> 异步执行，立即返回 <code>{ok, started}</code>

第八部分：工程实践与踩坑档案

这是全项目最有价值的资产。注释几乎不写「这段代码在做什么」，只写「为什么不能那样做」，且每条都标注完整链条：症状 → 根因 → 为什么本地测不出来 → 为什么选这个方案。

8.1 三类「本地测不出」的问题

类别一：Workers isolate 生命周期

问题	症状	位置
<code>waitUntil</code> 未挂 → 静默丢弃	有请求数、有延迟， <code>token / credits / msg</code> 全 0，且无报错	<code>proxy.ts</code> / <code>router.ts</code> / <code>apikeys.ts</code>
模块顶层 I/O → Worker 启动即崩	<code>Disallowed operation called within global scope</code>	<code>ids.ts</code>
<code>request.signal</code> 不触发 → <code>inFlight</code> 只增不减	「之前能用、用着用着全空」	<code>proxy.ts</code>
占位 INSERT 压进 TTFB	首字时间被 D1 往返吃掉	<code>d1.ts</code>

`waitUntil` 的完整说明（最重要的一段）：

⚠ 这不是可选优化，而是流式用量能不能落库的决定性一环。Cloudflare Workers 的语义是：**handler 返回 Response 后，只有被 `ctx.waitUntil` 显式延寿的 promise 才保证跑完，其余会在 isolate 回收时被静默丢弃**。而流式请求的收尾天生躲在这个空档里——Response 立刻返回给客户端，用量回填却发生在上游流读完之后的 `tap.flush` 中；那句 UPDATE 落进 `unprotected` 窗口就会被吞掉。**本地 vitest 完全测不出这个问题（Node 没有 isolate 回收），Go 版更是天然免疫**——它跑在常驻进程里。移植到 Workers 时这一环极易漏掉。

Hono `c.executionCtx` 的陷阱：

```
function waitUntilOf(c: any): (p: Promise<unknown>) => void {
  let ctx: ExecutionContext | undefined;
  try {
    ctx = c.executionCtx as ExecutionContext | undefined;
  } catch {
    ctx = undefined; // ⚠ 这个 try 不是防御性冗余，是必需的
  }
  ...
}
```

Hono 的 `c.executionCtx` 是 **getter**，在没有 `ExecutionContext` 的宿主下它会 `throw new Error("This context has no ExecutionContext")`，而不是优雅地返回 `undefined`。那么 `c?.executionCtx` 里的可选链救不了——它只对 `undefined/null` 生效，对抛异常无效。

类别二：跨区域网络

问题	数据	位置
D1 单区域跨洋	同一操作 123ms ↔ 247ms 反复横跳	<code>apikeys.ts</code> / <code>config.ts</code>
三处 KV 读都在 100ms 量级	配置 / 降级门 / 模型目录，且全是每请求必读	<code>config.ts</code>

对策矩阵：

读点	原策略	新策略
配置	5s TTL（间隔 >5s 就 miss）	30s 新鲜 + 5min SWR
降级门	完全无缓存 （稳定 100ms，一次不落）	加内存缓存 + TTL
模型目录	60s 快照（周期性 miss）	快照 + 缓存探针
子密钥	每请求查 D1	30s 内存缓存 + 负缓存

并行化机会（源码明确列出）：

冷启动优化：配置（1 次 KV）与子密钥（1 次 D1）**互不依赖**，原先串行 `await` 等于把两段 RTT 相加。这里并行发起，总耗时取较慢者。`prompt` 与 `models` 是两次**互不相干**的 KV 读，原先串行 `await`，于是两笔 ~100ms 的往返首尾相接叠成 ~200ms。

类别三：无状态导致的降级

问题	对策
模块级 promise 缓存随 isolate 回收	版本门（1 次 KV 读替代 14 次 D1）
后台刷新 promise 被回收 → <code>ts</code> 不更新	<code>CONFIG_STALE_MS</code> 留 5min 大窗口
<code>inFlight</code> 软计数泄漏	三重回收 + 选号放宽

8.2 具体的真机 bug 修复档案

Bug 1: CORS 头从未出现在成功响应上

```
// △ CORS 头必须在 `await next()` **之后**写进 c.res.headers
app.use("", async (c, next) => {
  c.set("wb2aT0", now());
  await next();
  const h = c.res?.headers;
  ...
});
```

Hono 的 `Context.set res` 里有一句 `this.#preparedHeaders = undefined`：任何在 `next()` 之前通过 `c.header()` 攒下的头，都会在 handler 交出 Response 的那一刻被丢弃。而本项目的 handler 一律 `return new Response(...)`（不是 `c.json / c.body`），走的正是不经过 `preparedHeaders` 合并的那条路——于是这些头从来没出现在成功响应上。

症状很有迷惑性：鉴权失败等走 `c.json` 的路径 CORS 正常，成功路径却没有；而同源面板看不出任何问题，只有跨域浏览器客户端（Web UI 直连 /v1）会被拦。

Bug 2: `ids.ts` 模块顶层 I/O 导致 Worker 启动即崩

`const deriveSalt = newMessageID()` 在模块顶层执行，Workers 明令禁止全局作用域 I/O，导致 Worker 启动即崩（`Disallowed operation called within global scope`）。单元测试跑不出来（Node 无此限制），只有真机/本地 Pages 才暴露。已改惰性初始化。

Bug 3: `oauthState` 不检查 HTTP 状态与业务 code

只解 `{code,msg,data}` 信封、不看 `res.ok`，上游 400 被吞成空 `state`，调用方只看到「没拿到 state」，与网络故障完全无法区分。同时缺 Go `commonHeaders` 的 `Origin / Referer / X-Requested-With` 与 CLI 专用 UA —— 缺 `Origin` 上游直接 400，登录功能 100% 失效。两者都已修，补了 7 个回归测试。

Bug 4: SSE 死循环导致 Worker 内存爆掉

详见 4.5.2 坑 1。

Bug 5: `global.chat_base` 误填国内域

早期这里误填国内域（从 CN 段复制未改），非空默认值把 `basesFor` 的 `cfg.global.x || GLOBAL.x` 短路——国际版账号的 chat/账单请求全被打到国内域，面板明明显示「国际版」却一直 401。且 `mergeDeep` 下 KV 的 `stored` 覆盖默认，只改默认值救不了已部署实例 → 加了 `BAD_GLOBAL_BASES` 一次性迁移。

Bug 6: 流式请求只 acquire 从不 release

流式请求此前只 acquire 从不 release, `inFlight` 只增不减, 几次请求后所有账号在途占满 → `inFlightFull` → `no_healthy_account` (真机「之前能用、用着用着全空」的根因)。正常结束走 `onEnd`; 客户端中途断连时 `tap.flush` 不触发, 改由 `request.signal` 兜底。 `released` 标志避免双重回收。

Bug 7: `touchKey` 在 isolate 回收时静默丢写

Touch 最佳努力: 统计口径不该因为写失败而拦住业务请求。挂 `waitUntil` 而非裸 `void` —— 否则响应返回、isolate 被回收时会把这笔 D1 写静默吞掉, 症状就是 `last_used` / `last_ip` 偶尔不更新 (只在 Workers 上出现, 本地/Go 复现不了)。

8.3 工程习惯总结

习惯	体现
纯函数优先	<code>pool-core.ts</code> / <code>autoroute.ts</code> / <code>usage-agg.ts</code> / <code>timing.ts</code> 全部无 IO, 为的是能单测
注释写「为什么不能那样做」	不写「这段在做什么」, 只写踩坑
显式标注「本地测不出」	反复出现「vitest 测不出 / Go 天然免疫」
兼容性兜底	旧版 engine 无 acquire 合并时补发; 0001 老行缺列时兜 0/"
降级不失败	版本门失效照跑迁移; KV 读失败用旧值; R2 未绑降级日志归档
安全口径不靠 TTL	管理面改动走 <code>invalidateKeyCache</code> 即时路径
幂等贯穿	迁移幂等、任务动作幂等、 <code>fill-ids</code> 幂等

第九部分：测试与质量保障

9.1 测试策略

类型	文件数	工具	覆盖对象
纯函数单测	8	Vitest	pool-core / autoroute / timing / classify / usage-agg
Miniflare 集成	6	Vitest + Miniflare	pool / alarms / migrate / oauth
契约守卫	3	Vitest	panel-keys-contract / panel-auth-split / frontend-syntax
生命周期	1	Vitest	workers-lifecycle
其他	15	Vitest	sse / proxy / apikeys / tasks / catalog / prompt / credits ...

9.2 测试命名反映的关注点

测试命名直接对应**历史故障**——是**回归守卫**而非覆盖率装饰：

workers-lifecycle.test.ts	← isolate 生命周期 (waitUntil / 全局作用域)
ttft-cache.test.ts	← 冷启动观测 (migrate 埋点、X-Cold-Start 落地)
upstream-timeouts.test.ts	← 三段超时 (fetch resolve 时机坑)
panel-auth-split.test.ts	← admin_key / api_key 分离
panel-keys-contract.test.ts	← 前后端字段契约 (前端零改动的保障)
logs-metrics-reset.test.ts	← 日志与用量重置
kv-reads.test.ts	← KV 读缓存行为
frontend-syntax.test.ts	← 前端语法 (原样拷贝的守卫)
fingerprint.test.ts	← 指纹脱敏
log-retention.test.ts	← 日志保留策略

9.3 `pool-core.test.ts` 用例清单 (32 例)

这份清单基本就是**选号算法**的规格说明：

成本分层

- └─ tier 判定：实测免费=0，无观测=1，实测收费=2，过期观测回落到 1
- └─ 硬过滤只留最优层：tier1（无观测）优先于 tier2（已实测收费）
- └─ tier0 存在时全部选 tier0（免费优先）
- └─ 条件探索：tier0 垄断 + 存在 tier1 + 超窗口 → 本次切 tier1-only
- └─ 探索窗口按（realm, model）独立分桶
- └─ cost_explore_interval=0 时关停探索

积分保底

- └─ 触底号接收费模型被拦；免费模型放行
- └─ 无本地观测时用目录倍率兜底判收费（堵住「无观测 = 放行」漏洞）
- └─ 倍率未知（目录外/内部模型）保守放行；倍率 0（限免）放行
- └─ 余额未触底不拦（保底只拦触底号）
- └─ 保底在选号里生效：触底号不会被选去接收费模型

模型级冷却

- └─ 6004 模型级冷却只避让该模型，其他模型仍可选（每模型独立）
- └─ auditOnly 条目只进台账不影响路由
- └─ 过期的模型级冷却不再避让

健康判定

- └─ 冷却 / 熔断 / 降权三者任一未到期即不可选（并存不叠加）
- └─ 在途占满按 realm 分档
- └─ 硬冷却识别（余额耗尽号）
- └─ expiry 三截止取最早；fallbackKind 区分熔断与软冷却
- └─ 快过期批次识别（prefer_expiring 的前提）

选号

- └─ exclude 命中的号被跳过（请求级轮换）
- └─ realm 过滤：global 模型只路由 global 账号
- └─ minPickGap：窗口内刚用过的号被挤向其他候选
- └─ top5 全刚用过 → LRU 兜底按 usedSeq 取最旧者（与时间精度无关）
- └─ 全冷却兜底：取到期最早者，排除禁用与硬冷却
- └─ 全冷却兜底也受积分保底约束（触底号不被捞回来接收费模型）
- └─ 全禁用时返回 null（上层映射 no_healthy_account）
- └─ model 为空时不做成本分层（无模型上下文）
- └─ 加权分布：闲置久的号被选中的次数明显更多

realm 隔离

- └─ realm 不匹配时本 realm 选不到，但 realm="" 时可放宽
- └─ realm 匹配时仍严格隔离：cn 请求不应选到 global 账号

在途软计数

- └─ 单账号在途占满仍可被选中（inFlight 是软计数，不能当硬门槛）
- └─ 多账号高负载：优先选空闲号，全满时才放宽

第十部分：设计取舍总表

10.1 关键参数及其「经营含义」

参数	值	技术含义	经营含义
KEY_CACHE_TTL_MS	30s	子密钥内存缓存时长	这个窗口内愿多放行多少配额
CONFIG_FRESH_MS	30s	配置新鲜窗口	改配置后最长多久全员生效（面板保存走即时失效）
CONFIG_STALE_MS	5min	配置可用上界	愿用多旧的配置（比全站 5xx 好）
MIN_PICK_GAP	100ms	防撞号窗口	—
MODEL_COST_TTL	6h	成本观测有效期	覆盖「夜间免费」时段优惠，但不跨时段生效
ACQUIRE_TTL_MS	10min	在途超时回收	—
EXPIRING_VIRTUAL_SLOTS	3	快过期号虚拟实例倍数	优先烧掉将过期额度
MP_CHAT_EVENT_GAP_MS	45s	小程序对话事件间隔	避开上游反作弊（数秒级连发会被整体回滚）
cache 各 TTL	30s/60s	缓存时长	同上

10.2 技术选型取舍

决策	选择	放弃的方案	理由
账号池	单实例 DO	多实例分片	顺序单线程 = 天然锁；几十~几百账号够用
迁移	版本门 + batch	每次全跑 DDL	14 次 D1 往返 → 1 次 KV 读
用量聚合	Worker 内线性扫描	SQL GROUP BY	口径写一遍，避免「改一处漏三处」
日志写入	流末一次写清	占位 + UPDATE	少一次 TTFB 阻塞 + 少一行写入额度 + 少一次静默丢弃风险
面板前端	原生 JS 零框架	React/Vue	原样拷贝即可零改动
子密钥判定	内存缓存	每请求查 D1	消除跨洋抖动（代价是配额陈旧窗口）
CLI	瘦客户端	本地遍历目录	Workers 下没有目录，强行保留只会空转
内容拦截	不罚号	换号重试	换任何号都撞同一审核
404 冷却	固定 60s	随 soft_rate 退避	偶发路径缺失不是限流信号
R2	可选	必选	建桶要绑支付方式

决策	选择	放弃的方案	理由
session_dead	连续 3 次才禁用	一次即禁	一次失败即禁用会误杀健康账号

10.3 「宁可…也不…」清单

这个项目有一组非常一致的价值取向：

宁可	也不
多跑一次幂等迁移	漏迁移
用 5 分钟前的配置	读不到配置就全站 5xx
下次再查一次 D1	让网络抖动变成 401 权限判决
显示 <code>note;dur=0</code>	让人以为埋点丢了
输出空计时头	输出只有 <code>total;dur=0</code> 的噪声
保守放行未知倍率模型	拦掉内部/别名模型导致号永久失联
让 <code>acquiredAt</code> 缺失也回收	泄漏计数永远卡死
记 0 而非删除段	让人怀疑是埋点漏了

第十一部分：架构张力与遗留问题

11.1 核心张力：跨 Worker RPC 成本

`PoolDO` 部署在独立 Worker 上，导致每次账号池交互都是一次完整跨 Worker HTTP 往返（真机实测 `pick` 段 41ms 量级）。

整个项目有大量优化是在减少 RPC 次数而非优化单次 RPC：

优化	做法
<code>pick</code> 合并	<code>acquire: 1</code> + <code>freshAuth</code> 内联
<code>note</code> 批量化	<code>kinds: string[]</code> 一次施加多个事件
<code>release</code> 合并	带 <code>cost</code> 一次完成记成本 + 释放在途
<code>note</code> 移出关键路径	成功路径的 <code>note</code> 挂 <code>waitUntil</code> ，移出而不是加快
兼容性兜底	旧 engine 无 <code>acquired</code> 时补发一次 <code>acquire</code> （防止并发闸门静默失效）

这是平台硬约束（Pages 不能自带 DO）传导下来的连锁设计变形，也是本架构最主要的性能天花板。

11.2 `total` 与各段之和的差值

`total` 恒 = 生成时刻 - `t0`，不是各段之和。差值包含：

- Workers 调度开销
- CORS 中间件
- 路由匹配
- 未计段的收尾工作

差值大 = 计时漏段了 —— 这个差值本身被当作诊断信号使用。

11.3 已识别的残余风险

风险	说明	现状
配额超额放行窗口	子密钥缓存 30s 内配额判定是陈旧的	已权衡为「一次对话的量级」
多 isolate 一致性	配置/密钥缓存各自为政	管理面走即时失效，其余最多滞后一个刷新周期

风险	说明	现状
流式日志丢失	客户端中途断连且 Worker 在 abort 回调前被回收	仅进程级硬中断会丢
DO 单实例容量	上千账号建议按 realm 拆两个 DO 实例	<code>AUDIT_AND_ARCHITECTURE.md</code> 第 9 节列出待确认
上游字段变动	任务中心 payload 按审计端点还原	集中在 <code>tasks.ts</code> 调整
<code>diagnose</code> 的 <code>pickable_but_unselected</code>	出现即说明兜底逻辑有 bug	已内置诊断信号

11.4 待确认的设计决策（来自审计文档）

`AUDIT_AND_ARCHITECTURE.md` 第 9 节列出的开放问题：

1. 账号规模与并发：单 PoolDO 实例能否覆盖？（几十~几百 OK；上千建议按 realm 拆两个 DO 实例）
2. Upstash：是否保留外部 Redis？（推荐纯 DO，当前已移除）
3. 定时任务：DO alarm vs 外部 cron？（当前三条路径并存）
4. 审计数据保留：D1 还是 R2？保留天数？（当前 7 天归档）

11.5 最有复用价值的五个文件

如果要借鉴这套工程实践，优先级从高到低：

排名	文件	可复用的方法论
1	<code>timing.ts</code>	TTFT 归因方法论：分段计时 + 单调时钟 + 固定顺序 + 双格式输出
2	<code>boot.ts</code>	冷启动成因分类：把「isolate 刚起来」与「cache TTL 到期」分开
3	<code>migrate.ts</code>	版本门 + 幂等迁移 + batch 压缩 + 失效即退化
4	<code>sse.ts</code>	单 TransformStream + 字符串快筛 + 增量聚合 + 三个具体坑
5	<code>apikeys.ts</code>	缓存 TTL 作为经营决策 + 负缓存 + 失败不缓存 + 管理面即时失效

11.6 一句话总结

workbuddy2api-plus-cf 是一份「真机踩坑档案」，而不是教科书式代码库。

它的注释几乎不解释代码在做什么，只解释为什么不能那样做；每条都带着完整的症状、根因、以及「为什么本地测不出来」。反复出现的三类问题 —— Workers isolate 生命周期、跨区域网络、无状态导致的降级 —— 恰好都是单元测试与本地环境天然覆盖不到的盲区，这也是这份档案最稀缺的价值所在。

文档基于 `main` 分支 @ `fe0867d` 的源码分析生成，所有技术细节均可回溯至具体文件与行号。